Symbolic Knowledge Discovery

FCA and pattern mining for actionable knowledge discovery

Miguel Couceiro (Part I) & Egor Dudyrev (Part II)

M2 NLP&SC 2025-26

IST, U.Lisbon, INESC-ID & IDMC, U.Lorraine, CNRS, Inria, LORIA

Based on references and notes by M. Kaytoue, S. Kuznetsov, A. Napoli, Sergei Obiedkov

Motivation & some mathematical background

Example I: A scientific question

Understanding the olfactory system

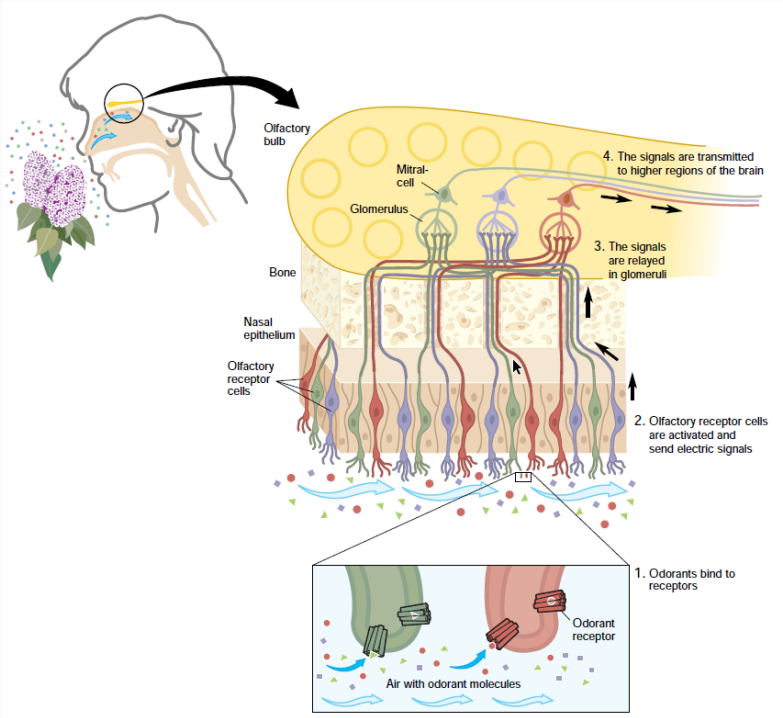
- Olfaction: ability to perceive odors
- Complex phenomenon from molecule to perception (1991)

Challenges:

- Links between physico-chemical properties and olfactory qualities (2004,2017)
- Difficulties to formulate rules

Impact:

- Fundamental neuroscience research
- Industry (food, perfume)
- Health (anosmia, ...)
- NLP (text/topic classification)



Example I: Smell, a luxury?

Presse CNRS Info Emploi Annuaires Intranet la fondation

Le CNRS La Recherche Innovation International

Accueil > CNRS Info

L'odorat, victime du COVID-19 à l'étude

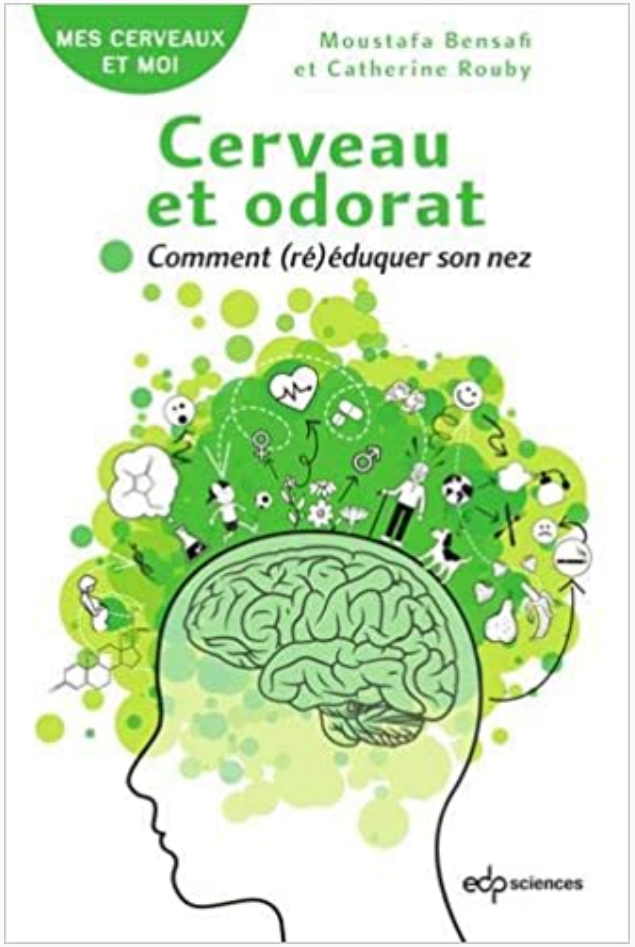
20 avril 2020 RECHERCHE

© goanovi / stock.adobe.com Partager

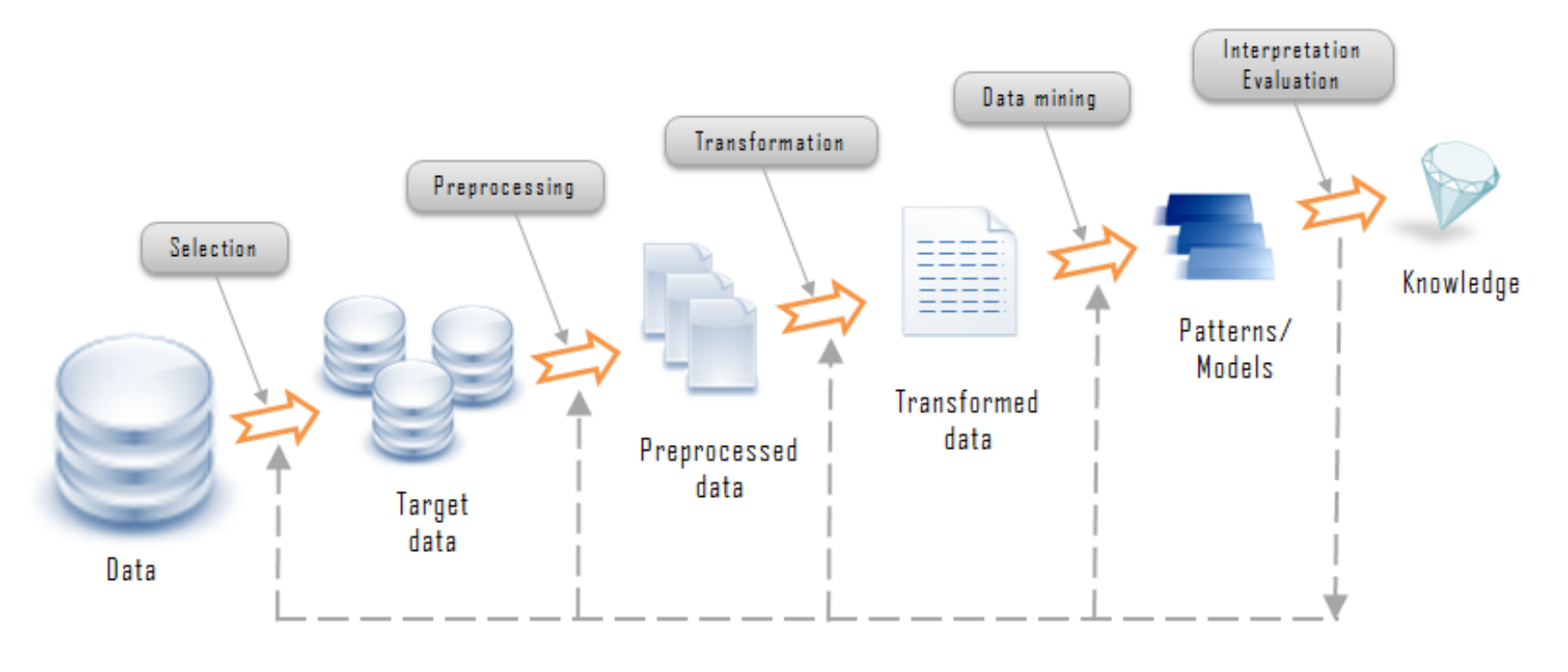
Certains patients atteints du COVID-19 ont signalé une perte de l'odorat. En partenariat avec d'autres structures françaises, le Centre de recherche en neurosciences de Lyon lance un site internet d'information et invite le grand public à participer à une étude sur ce sens méconnu. Les neuroscientifiques Moustafa Bensafi et Camille Ferdenzi, du Centre de recherche en neurosciences de Lyon, répondent à nos questions.

A - / A+

Partager ce contenu



Eliciting hypotheses from data: A KDD task



The KDD process is iterative, interactive and guided by an analyst

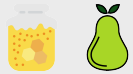
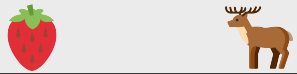
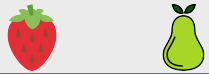
Eliciting hypotheses from data: A KDD task

Data collection

- Descriptions: chemical properties, molecular structure, ...(2005)
- Targets: odor(s), valence...(1994)

Mining (a few and good) hypotheses (in a large search space)

- Clusters, concepts, association rules, redescriptions, ...
- Mining subgroups discriminating a target attribute (1997)
  Data query: $s = nAT \geq 24 \wedge nC \leq 11$
  Query result: $support(s) = \{24, 48, 1633\}$
  $Quality(s \rightarrow pear)$ is high: all the pears, only the pears

ID	MW	nAT	nC	Odor
1	150.19	21	11	
24	128.24	29	11	
48	136.16	24	9	
60	152.16	23	11	
82	151.28	27	12	
1633	142.22	27	10	

This course...

In our investigations:

- What do we mine?
- How do we mine the best patterns?
- For what purpose?

Evaluation: Project on symbolic knowledge discovery & RAG-like tools (i.e., KG construction, ontology design)

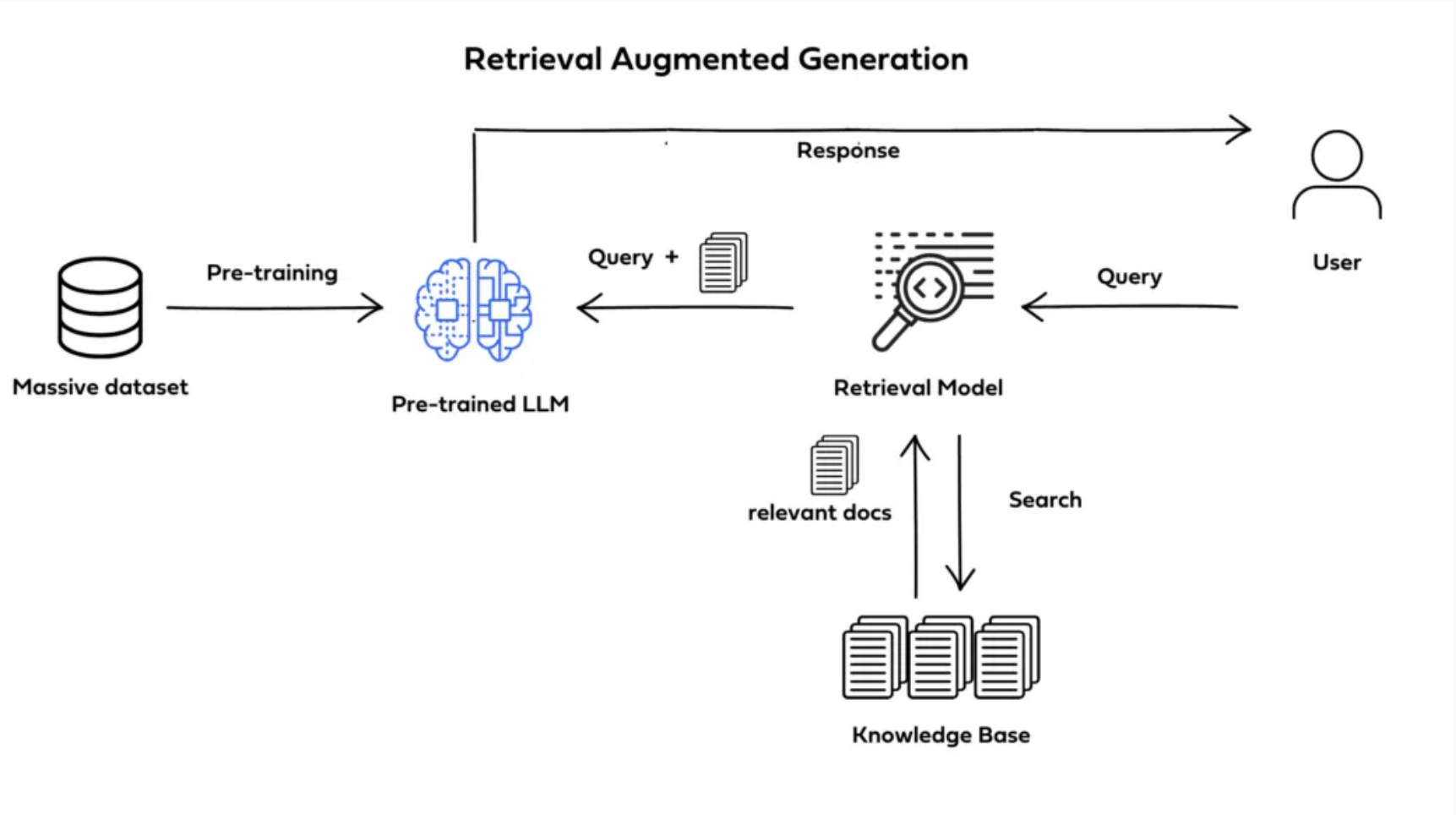
This course...

In our investigations:

- What do we mine?
- How do we mine the best patterns?
- For what purpose?

Evaluation: Project on symbolic knowledge discovery & RAG-like tools (i.e., KG construction, ontology design)

Retrieval-Augmented Generation (RAG)



From https://medium.com/@krtarunsingh/

introduction-to-retrieval-augmented-generation-rag-and-its-transformative-role-in-ai-c07e35da7f01

Case study

Objective: Enrich a domain-specific KG with symbolic data and implement a GraphRAG pipeline

1. Find a dataset of your interest (tabular data)
2. Convert your dataset into turtle format (KG)
3. Use this graph representation when querying your data with graphRAG. Comment your results.
4. Now find concepts and implications using the tools seen in the course
5. Integrate them in your KG (we call this enriched KG "ontology")
6. Use this ontology for querying your data with GraphRAG. Comment your results.
7. Propose an experimental setting (questions, metrics, etc.) to compare both approaches: KG vs. Ontologies!

Format: Work in groups (at most 3 students per group)

Case study

Objective: Enrich a domain-specific KG with symbolic data and implement a GraphRAG pipeline

1. Find a dataset of your interest (tabular data)
2. Convert your dataset into turtle format (KG)
3. Use this graph representation when querying your data with graphRAG. Comment your results.
4. Now find concepts and implications using the tools seen in the course
5. Integrate them in your KG (we call this enriched KG "ontology")
6. Use this ontology for querying your data with GraphRAG. Comment your results.
7. Propose an experimental setting (questions, metrics, etc.) to compare both approaches: KG vs. Ontologies!

Format: Work in groups (at most 3 students per group)

Contacts:

- Miguel COUCEIRO (Part I):
  Email: `miguel.couceiro@loria.fr` or `miguel.couceiro@inesc-id.pt`
  WebPage: `https://members.loria.fr/mcouceiro/`
- Egor DUDYREV (Part II):
  Email: `egor.dudyrev@loria.fr`,
  WebPage: `https://egordudyrev.github.io`

References

- L. Buck. A novel multigene family may encode odorant receptors: A molecular basis for odor recognition. Cell (1991)
- U. J. Meierhenrich et al. The Molecular Basis of Olfactory Chemoreception. Angewandte Chemie International Edition 43.47 (2004)
- A. Keller et al. Predicting human olfactory perception from chemical features of molecules. Science(2017)
- I. V. Tetko et al. Virtual Computational Chemistry Laboratory - Design and Description. Journal of Computer-Aided Molecular Design 19.6 (2005)
- S. Arctander. Perfume and flavor materials of natural origin. Vol. 2. 1994
- S. Wrobel. An Algorithm for Multi-relational Discovery of Subgroups. PKDD. 1997
- Wenqi Fan, *et al.*: A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. KDD 2024: 6491-6501

Some mathematical background:
Ordered sets and lattices

Some mathematical background: orders and ordered sets

Let P be a set. An partial order on P is a binary relation $\leq$ on P such that:

- $x \leq x$ (reflexivity),
- $x \leq y$ and $y \leq x$ imply $x = y$ (antisymmetry),
- $x \leq y$ and $y \leq z$ imply $x \leq z$ (transitivity), for every $x, y, z \in P$.

If $\leq$ is reflexive and transitive, then it is called a quasi-order.
A linear order is a partial order $\leq$ such that, for every $x, y \in P$, $x \leq y$ or $y \leq x$.

An equivalence is a relation $\equiv$ that is reflexive, symmetric and transitive.

A set equipped with a partial order (resp. quasi-order, linear order) is said to be a partially ordered set or, simply, a poset (resp. quasi-, linearly ordered set).

NB: Every quasi-order $\leq$ induces the equivalence: $x \equiv y$ iff $x \leq y$ and $y \leq x$.

Example: For any set X, the powerset $2^X = \mathcal{P}(X)$ of all subsets of X is a poset with the partial ordering $\leq$ defined by: for $A, B \subseteq X$, $A \leq B$ iff $A \subseteq B$.

Some mathematical background: orders and ordered sets

Let P be a set. An partial order on P is a binary relation $\leq$ on P such that:

- $x \leq x$ (reflexivity),
- $x \leq y$ and $y \leq x$ imply $x = y$ (antisymmetry),
- $x \leq y$ and $y \leq z$ imply $x \leq z$ (transitivity), for every $x, y, z \in P$.

If $\leq$ is reflexive and transitive, then it is called a quasi-order.
A linear order is a partial order $\leq$ such that, for every $x, y \in P$, $x \leq y$ or $y \leq x$.

An equivalence is a relation $\equiv$ that is reflexive, symmetric and transitive.

A set equipped with a partial order (resp. quasi-order, linear order) is said to be a partially ordered set or, simply, a poset (resp. quasi-, linearly ordered set).

NB: Every quasi-order $\leq$ induces the equivalence: $x \equiv y$ iff $x \leq y$ and $y \leq x$.

Example: For any set X, the powerset $2^X = \mathcal{P}(X)$ of all subsets of X is a poset with the partial ordering $\leq$ defined by: for $A, B \subseteq X$, $A \leq B$ iff $A \subseteq B$.

Some mathematical background: orders and ordered sets

Let P be a set. An partial order on P is a binary relation $\leq$ on P such that:

- $x \leq x$ (reflexivity),
- $x \leq y$ and $y \leq x$ imply $x = y$ (antisymmetry),
- $x \leq y$ and $y \leq z$ imply $x \leq z$ (transitivity), for every $x, y, z \in P$.

If $\leq$ is reflexive and transitive, then it is called a quasi-order.
A linear order is a partial order $\leq$ such that, for every $x, y \in P$, $x \leq y$ or $y \leq x$.

An equivalence is a relation $\equiv$ that is reflexive, symmetric and transitive.

A set equipped with a partial order (resp. quasi-order, linear order) is said to be a partially ordered set or, simply, a poset (resp. quasi-, linearly ordered set).

NB: Every quasi-order $\leq$ induces the equivalence: $x \equiv y$ iff $x \leq y$ and $y \leq x$.

Example: For any set X, the powerset $2^X = \mathcal{P}(X)$ of all subsets of X is a poset with the partial ordering $\leq$ defined by: for $A, B \subseteq X$, $A \leq B$ iff $A \subseteq B$.

Some mathematical background: Upper and lower bounds

Let P be an ordered set and let $S \subseteq P$. An element $x \in P$ is

- an upper bound of S if $s \leq x$, for all $s \in S$.
- a lower bound of S if $x \leq s$, for all $s \in S$.

The set of all upper bounds of S is denoted by S^u ("S upper"):

$$S^u := \{x \in P | (\forall s \in S)\, s \leq x\}$$

The set of all lower bounds of S is denoted by S^ℓ ("S lower"):

$$S^\ell := \{x \in P | (\forall s \in S)\, s \geq x\}$$

Some mathematical background: Suprema and infima

Let P be an ordered set and let $S \subseteq P$. An element $x \in P$ is called the

- least upper bound of S (or supremum) of S if for every $y \in S^u \setminus \{x\}$, $x \leq y$. For $x, y \in P$, their supremum is usually denoted by $x \vee y$ (the **join**).

  NB: if it exists, it is unique!

- More generally, the supremum of $S \subseteq P$ (when it exists) is denoted by $\bigvee S$ (join of S).

- greatest lower bound of S (or infimum) of S if for every $y \in S^\ell \setminus \{x\}$, $y \leq x$. For $x, y \in P$, their infimum is usually denoted by $x \wedge y$ (the **meet**).

  NB: if it exists, it is unique!

- More generally, the infimum of $S \subseteq P$ (when it exists) is denoted by $\bigwedge S$ (meet of S).

Some mathematical background: Suprema and infima

Let P be an ordered set and let $S \subseteq P$. An element $x \in P$ is called the

- least upper bound of S (or supremum) of S if for every $y \in S^u \setminus \{x\}$, $x \leq y$. For $x, y \in P$, their supremum is usually denoted by $x \vee y$ (the **join**).

  NB: if it exists, it is unique!

- More generally, the supremum of $S \subseteq P$ (when it exists) is denoted by $\bigvee S$ (join of S).

- greatest lower bound of S (or infimum) of S if for every $y \in S^\ell \setminus \{x\}$, $y \leq x$. For $x, y \in P$, their infimum is usually denoted by $x \wedge y$ (the **meet**).

  NB: if it exists, it is unique!

- More generally, the infimum of $S \subseteq P$ (when it exists) is denoted by $\bigwedge S$ (meet of S).

Some mathematical background: Lattices and complete lattices

A lattice is a poset L s.t. for every pair $x, y \in P$, both $x \vee y$ and $x \wedge y$ exist.

A lattice L is said to be complete if both $\bigvee S$ and $\bigwedge S$ exist, for every $S \subset L$.

Example: The powerset $2^X = \mathcal{P}(X)$ endowed with $\subseteq$ is a (complete) lattice.

Lattices as algebras: A lattice is an algebraic structure $\langle L; \vee, \wedge \rangle$ such that

- the binary operations $\vee$ and $\wedge$ are commutative and associative, and
- for every $x, y \in L$, $x \wedge (x \vee y) = x = x \vee (x \wedge y)$.

Exercise. Let $\langle L; \vee, \wedge \rangle$ be a lattice. Show that both $\vee$ and $\wedge$ are idempotent, and that the relations defined by

- $x \leq_1 y$ iff $x \vee y = y$
- $x \leq_2 y$ iff $x \wedge y = x$

are partial orders. Are they the same?

Some mathematical background: Lattices and complete lattices

A lattice is a poset L s.t. for every pair $x, y \in P$, both $x \vee y$ and $x \wedge y$ exist.

A lattice L is said to be complete if both $\bigvee S$ and $\bigwedge S$ exist, for every $S \subset L$.

Example: The powerset $2^X = \mathcal{P}(X)$ endowed with $\subseteq$ is a (complete) lattice.

Lattices as algebras: A lattice is an algebraic structure $\langle L; \vee, \wedge \rangle$ such that

- the binary operations $\vee$ and $\wedge$ are commutative and associative, and
- for every $x, y \in L$, $x \wedge (x \vee y) = x = x \vee (x \wedge y)$.

Exercise. Let $\langle L; \vee, \wedge \rangle$ be a lattice. Show that both $\vee$ and $\wedge$ are idempotent, and that the relations defined by

- $x \leq_1 y$ iff $x \vee y = y$
- $x \leq_2 y$ iff $x \wedge y = x$

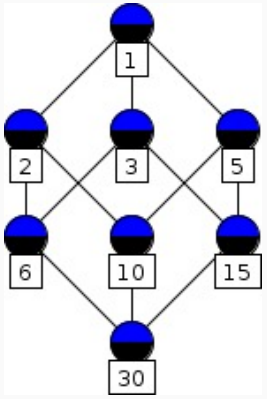
are partial orders. Are they the same?

Further examples of lattices

$\mathbb{N}$ with the divisibility relation is a lattice:

- $x \leq y$ iff y is a divisor of x as integers (Euclidean division).
- $x \vee y = gcd(x, y)$ (greatest common divisor)
- $x \wedge y = lcm(x, y)$ (least common multiple)

The lattice of the divisors of 30.



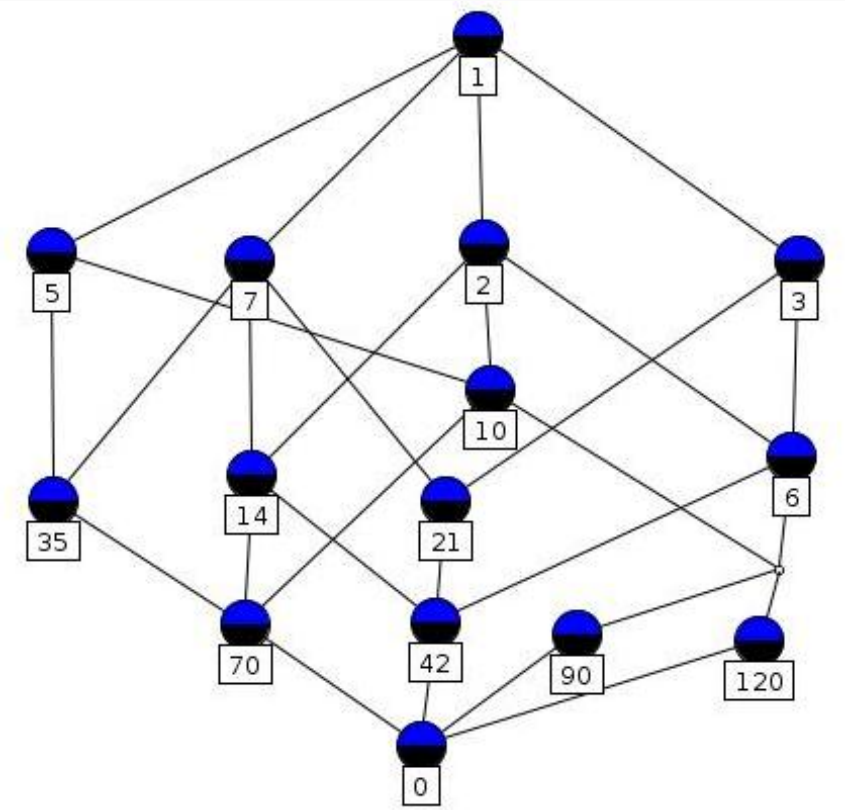
An example of classification-based reasoning

The classification process of a concept C or an individual $\mathbf{x}$ in a concept hierarchy relies on three main steps:

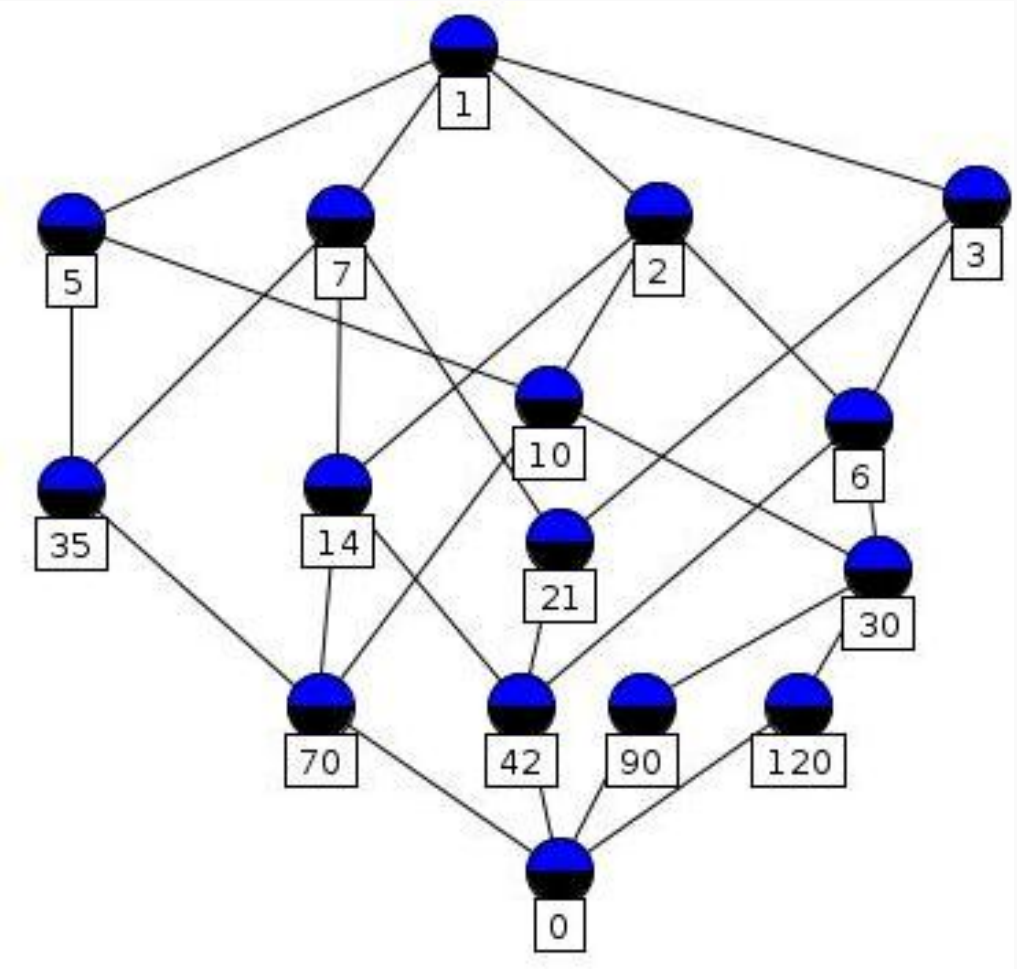
- search for the most specific subsumers (MSS) of $\mathbf{x}$,
- search for the most general subsumees (MGS) of $\mathbf{x}$
  (this step is not necessary for an individual),
- insertion of $\mathbf{x}$ in the concept hierarchy, under its MSS and over its MGS.

Classification in the division poset

Problem: Classify 30 in the following division poset.



Classification in the division poset



Outline

1. Motivation & some mathematical background
2. FCA & Algorithms
3. Implications & functional dependencies
4. Association rules & hypotheses

PART I

FCA & Algorithms

Formal Concept Analysis: motivation

Formal Concept Analysis (FCA) is a mathematical formalism based on lattice theory, classification and concept discovery.

FCA follows a human centered approach and supports exploration operations through the concept lattice:

- **Discovery of concepts**, i.e. classes of individuals with a description.
- **Organization of concepts** into a poset based on a subsumption relation.
- It supports exploration, e.g. information retrieval, visualization, ...

The FCA process

- The basic procedure of FCA is based on a simple representation of data, i.e. a binary table called a formal context.
- Each formal context is transformed into a mathematical structure called concept lattice.
- The information contained in the formal context is preserved.
- The concept lattice is the basis for data analysis.

Graphical representation: analysis, mining, visualization, interpretation. . .

Formal Contexts

Let G and M be sets. Elements of G are called **objects**, elements of M are called **attributes**.

Let $I \subseteq G \times M$ be a binary relation. If $(g, m) \in I$, means that object g has attribute m. The triple $\mathbb{K} \stackrel{\text{def}}{=} (G, M, I)$ is called a **(formal) context**.

Example: 2D figures

	G \ M	a	b	c	d
1		×			×
2		×		×	
3			×	×	
4			×	×	×

Galois connections and derivations

Given $\mathbb{K} := (G, M, I)$ consider two mappings $\varphi\colon 2^G \to 2^M$ and $\psi\colon 2^M \to 2^G$:

$$\begin{aligned} \varphi(A) &\stackrel{\text{def}}{=} \{m \in M \mid gIm \ \text{ for all } g \in A\}, \\ \psi(B) &\stackrel{\text{def}}{=} \{g \in G \mid gIm \ \text{ for all } m \in B\}. \end{aligned}$$

These mappings are usually called derivations.

Properties: for all $A_1, A_2 \subseteq G$, $B_1, B_2 \subseteq M$:

1. $A_1 \subseteq A_2 \Rightarrow \varphi(A_2) \subseteq \varphi(A_1)$ (order-reversing)
2. $B_1 \subseteq B_2 \Rightarrow \psi(B_2) \subseteq \psi(B_1)$ (order-reversing)
3. $A_1 \subseteq \psi(\varphi(A_1))$ and $B_1 \subseteq \varphi(\psi(B_1))$ (extensivity)

Thus: φ and ψ define a **Galois connection** between $(2^G, \subseteq)$ and $(2^M, \subseteq)$, and $A \subseteq \psi(B) \Leftrightarrow B \subseteq \varphi(A)$

Galois connections and derivations

Given $\mathbb{K} := (G, M, I)$ consider two mappings $\varphi\colon 2^G \to 2^M$ and $\psi\colon 2^M \to 2^G$:

$$\begin{aligned} \varphi(A) &\stackrel{\text{def}}{=} \{m \in M \mid gIm \ \text{ for all } g \in A\}, \\ \psi(B) &\stackrel{\text{def}}{=} \{g \in G \mid gIm \ \text{ for all } m \in B\}. \end{aligned}$$

These mappings are usually called derivations.

Properties: for all $A_1, A_2 \subseteq G$, $B_1, B_2 \subseteq M$:

1. $A_1 \subseteq A_2 \Rightarrow \varphi(A_2) \subseteq \varphi(A_1)$ (order-reversing)
2. $B_1 \subseteq B_2 \Rightarrow \psi(B_2) \subseteq \psi(B_1)$ (order-reversing)
3. $A_1 \subseteq \psi(\varphi(A_1))$ and $B_1 \subseteq \varphi(\psi(B_1))$ (extensivity)

Thus: φ and ψ define a **Galois connection** between $(2^G, \subseteq)$ and $(2^M, \subseteq)$, and $A \subseteq \psi(B) \Leftrightarrow B \subseteq \varphi(A)$

Properties of derivations $(\cdot)'$

In FCA: we unify φ and ψ into a single notation $(\cdot)'$

$$\begin{aligned} A' &\stackrel{\text{def}}{=} \{m \in M \mid gIm \ \text{ for all } g \in A\}, \\ B' &\stackrel{\text{def}}{=} \{g \in G \mid gIm \ \text{ for all } m \in B\}. \end{aligned}$$

Properties: For $\mathbb{K} = (G, M, I)$, $A, A_1, A_2 \subseteq G$, we have

- $A \subseteq A''$
- $A''' = A'$
- $(A_1 \cup A_2)' = A_1' \cap A_2'$;

Similar properties hold for subsets of attributes.

Properties of derivations $(\cdot)'$

In FCA: we unify φ and ψ into a single notation $(\cdot)'$

$$\begin{aligned} A' &\stackrel{\text{def}}{=} \{m \in M \mid gIm \ \text{ for all } g \in A\}, \\ B' &\stackrel{\text{def}}{=} \{g \in G \mid gIm \ \text{ for all } m \in B\}. \end{aligned}$$

Properties: For $\mathbb{K} = (G, M, I)$, $A, A_1, A_2 \subseteq G$, we have

- $A \subseteq A''$
- $A''' = A'$
- $(A_1 \cup A_2)' = A_1' \cap A_2'$;

Similar properties hold for subsets of attributes.

A first concrete and culinary example

Pizzas\Toppings	tomato	onion	oregano	mozzarella	palm heart
filadelfia		x	x		x
regina	x		x	x	
roma	x	x	x	x	
margherita	x			x	
napoli	x	x	x	x	
castelli	x		x	x	

Computing the images of sets of objects and attributes

{g2}' = {m1, m3, m4}:

G \ M	m1	m2	m3	m4	m5
g1		x	x		x
g2	**x**		**x**	**x**	
g3	x	x	x	x	
g4	x			x	
g5	x	x	x	x	
g6	x		x	x	

Computing the images of sets of objects and attributes

{m3}' = {g1, g2, g3, g5, g6}:

G \ M	m1	m2	m3	m4	m5
g1		x	**x**		x
g2	x		**x**	x	
g3	x	x	**x**	x	
g4	x			x	
g5	x	x	**x**	x	
g6	x		**x**	x	

Computing the images of sets of objects and attributes

{g3,g5}' = {m1, m2, m3, m4}:

G \ M	m1	m2	m3	m4	m5
g1		x	x		x
g2	x		x	x	
g3	**x**	**x**	**x**	**x**	
g4	x			x	
g5	**x**	**x**	**x**	**x**	
g6	x		x	x	

Computing the images of sets of objects and attributes

{m3,m4}' = {g2, g3, g5, g6}:

G \ M	m1	m2	m3	m4	m5
g1		x	x		x
g2	x		**x**	**x**	
g3	x	x	**x**	**x**	
g4	x			x	
g5	x	x	**x**	**x**	
g6	x		**x**	**x**	

Formal concepts

(Formal) concept is a pair (A, B) such that $A \subseteq G,\ B \subseteq M$ **and**

$$A' = B \quad \text{and} \quad B' = A$$

A is called the extent **and** B the intent of the concept (A, B).

Concepts are partially ordered by the relation

$$(A_1, B_1) \geq (A_2, B_2) \iff A_1 \supseteq A_2 \quad (B_2 \supseteq B_1).$$

This partially ordered set constitutes the concept lattice $\underline{\mathfrak{B}}(G, M, I)$

NB: Formal concepts correspond to maximal rectangles in contexts

Formal concepts

(Formal) concept is a pair (A, B) such that $A \subseteq G,\ B \subseteq M$ **and**

$$A' = B \quad \text{and} \quad B' = A$$

A is called the extent **and** B the intent of the concept (A, B).

Concepts are partially ordered by the relation

$$(A_1, B_1) \geq (A_2, B_2) \iff A_1 \supseteq A_2 \quad (B_2 \supseteq B_1).$$

This partially ordered set constitutes the concept lattice $\underline{\mathfrak{B}}(G, M, I)$

NB: Formal concepts correspond to maximal rectangles in contexts

Back to our example...

	G \ M	a	b	c	d
1		×			×
2		×		×	
3			×	×	
4			×	×	×

Objects:

1 – equilateral triangle

2 – rectangle triangle

3 – rectangle

4 – square

Attributes:

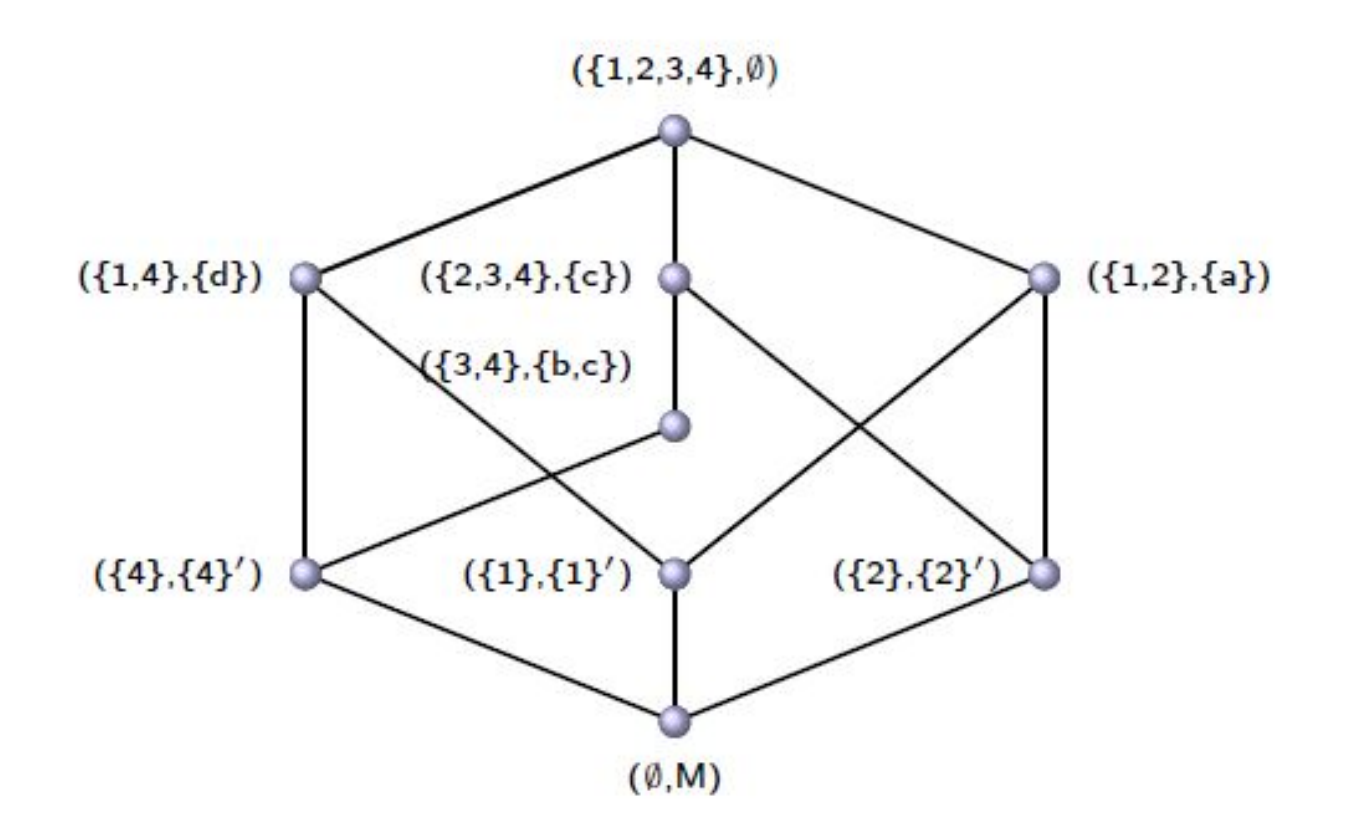
a – has 3 vertices

b – has 4 vertices

c – has a right angle

d – equilateral

The corresponding concept lattice



	G \ M	a	b	c	d
1		×			×
2		×		×	
3			×	×	
4			×	×	×

a – has 3 vertices

b – has 4 vertices

c – has a direct angle

d – equilateral

Closure systems...

Concept lattices are a noteworthy data science example of a closure system:

- a set of descriptions of some objects from a domain
- together with all possible intersections of such descriptions.

These intersections give a similarity of objects in terms of descriptions.

Closure operator on a set

A closure operator on set G is a mapping $\varphi\colon \mathcal{P}(G) \to \mathcal{P}(G)$ such that:

1. $\varphi(\varphi(X)) = \varphi(X)$ (idempotency)
2. $X \subseteq \varphi(X)$ (extensity)
3. $X \subseteq Y \Rightarrow \varphi(X) \subseteq \varphi(Y)$ (monotonicity)

For a closure operator φ the set φX is called closure of X.

A subset $X \subseteq G$ is called closed if $\varphi X = X$.

NB: Both $(\cdot)'' \colon 2^G \to 2^G$ and $(\cdot)'' \colon 2^M \to 2^M$ are closure operators.

The study of such closure systems is the main focus of FCA.

Closure operator on a set

A closure operator on set G is a mapping $\varphi\colon \mathcal{P}(G) \to \mathcal{P}(G)$ such that:

1. $\varphi(\varphi(X)) = \varphi(X)$ (idempotency)
2. $X \subseteq \varphi(X)$ (extensity)
3. $X \subseteq Y \Rightarrow \varphi(X) \subseteq \varphi(Y)$ (monotonicity)

For a closure operator φ the set φX is called closure of X.

A subset $X \subseteq G$ is called closed if $\varphi X = X$.

NB: Both $(\cdot)''\colon 2^G \to 2^G$ and $(\cdot)''\colon 2^M \to 2^M$ are closure operators.

The study of such closure systems is the main focus of FCA.

Closure operator on a set

A closure operator on set G is a mapping $\varphi\colon \mathcal{P}(G) \to \mathcal{P}(G)$ such that:

1. $\varphi(\varphi(X)) = \varphi(X)$ (idempotency)
2. $X \subseteq \varphi(X)$ (extensity)
3. $X \subseteq Y \Rightarrow \varphi(X) \subseteq \varphi(Y)$ (monotonicity)

For a closure operator φ the set φX is called closure of X.

A subset $X \subseteq G$ is called closed if $\varphi X = X$.

NB: Both $(\cdot)''\colon 2^G \to 2^G$ and $(\cdot)''\colon 2^M \to 2^M$ are closure operators.

The study of such closure systems is the main focus of FCA.

The basic theorem of FCA

$\underline{\mathfrak{B}}(G, M, I)$ is a complete lattice where the join and the meet are given by:

- $\bigvee_{k \in K}(A_k, B_k) = ((\bigcup_{k \in K} A_k)'', \bigcap_{k \in K} B_k)$
- $\bigwedge_{k \in K}(A_k, B_k) = (\bigcap_{k \in K} A_k, (\bigcup_{k \in K} B_k)'')$

NB: the intersection of closed sets is a closed set but the union is not necessarily a closed set.

A simple algorithm to construct concept lattices:

The algorithm of Bordat

(not covered)

From contexts to concept lattices

Let us consider a set of objects G and a set of attributes M.

- The Bordat algorithm is able to build the formal concepts and the order between the concepts, thus the whole concept lattice.
- Let us consider a set of objects G and a set of attributes M.
- Set the bottom concept as the pair $\perp = (M', M)$ unless there is one object owning all attributes (then $M' \neq \emptyset$).

The Bordat algorithm

Let $C_k = (X_k, Y_k)$ be a formal concept.

The concept $C_{k+1} = (X_{k+1}, Y_{k+1})$ is a subsumer of C_k in the lattice whenever:

(i) $X_{k+1} = X_k \cup \{x \in G \setminus X_k \ : \ f_{C_k}(x) = Y_{k+1}\}$ where f_{C_k} computes the intent of x restricted to Y_k

NB: extensions are growing, while intensions are shrinking!

(ii) $Y_{k+1} \in Max\{f_{C_k}(x) \ : \ x \notin X_k\}$

The Bordat algorithm

G \ M	a	b	c	d	e
g1		x	x		x
g2	x		x	x	
g3	x	x	x	x	
g4	x			x	
g5	x	x	x	x	
g6	x		x	x	

- $G = \{g1, g2, g3, g4, g5, g6\}$ and $M = \{a, b, c, d, e\}$
- $\perp = (\emptyset, M) = (X_0, Y_0)$

The Bordat algorithm

- $G = \{g1, g2, g3, g4, g5, g6\}$ and $M = \{a, b, c, d, e\}$
- $\perp = (\emptyset, M) = (X_0, Y_0)$

Finding the subsumers of $(X_0, Y_0) = (\emptyset, M)$:

- $X_1 = X_0 \cup \{x \in G \setminus X_0 \ : \ f_{C_0}(x) = Y_1\}$
- $Y_1 \in Max\{f_{C_0}(x) \ : \ x \notin X_0\}$
- $f_{C_0} \subseteq (G \setminus \emptyset) \times \{a, b, c, d, e\}$
- $\{f_{C_0}(x) \ : \ x \notin X_0\} = \{f_{C_0}(x) \ : \ x \in G\} =$
  $\{f_{C_0}(g1), f_{C_0}(g2), f_{C_0}(g3), f_{C_0}(g4), f_{C_0}(g5), f_{C_0}(g6)\} =$
  $\{\{b, c, e\}, \{a, c, d\}, \{a, b, c, d\}, \{a, d\}, \{a, b, c, d\}, \{a, c, d\}\}$

Possible intents Y_1:

- $Y_1 \in Max\{f_{C_0}(x) \ : \ x \notin X_0\} = \{\{b, c, e\}, \{a, b, c, d\}\}$

The Bordat algorithm

- $Y_1 = \{b, c, e\}$
  $X_1 = X_0 \cup \{x \in G \setminus X_0 \ : \ f_{C_0}(x) = Y_1\} =$
  $\{x \in G \ : \ f_{C_0}(x) = \{b, c, e\}\} = \{g1\}$
- $Y_2 = \{a, b, c, d\}$
  $X_2 = X_0 \cup \{x \in G \setminus X_0 \ : \ f_{C_0}(x) = Y_2\} =$
  $\{x \in G \ : \ f_{C_0}(x) = \{a, b, c, d\}\} = \{g3, g5\}$

Direct subsumers of $C_0 = (X_0, Y_0)$:

- $C_1 = (X_1, Y_1) = (\{g1\}, \{b, c, e\})$ and
- $C_2 = (X_2, Y_2) = (\{g3, g5\}, \{a, b, c, d\})$

The Bordat algorithm

Direct subsumers of $C_1 = (X_1, Y_1) = (\{g1\}, \{b, c, e\})$:

- $X_3 = X_1 \cup \{x \in G \setminus X_1 \ : \ f_{C_1}(x) = Y_3\}$
- $f_{C_1} : (G \setminus \{g1\}) \times \{b, c, e\}$
- $Y_3 \in Max\{f_{C_1}(x) \ : \ x \in \{g2, g3, g4, g5, g6\}\}$
- $Y_3 \in Max\{f_{C_1}(g2), f_{C_1}(g3) f_{C_1}(g4), f_{C_1}(g5), f_{C_1}(g6)\} =$ $Max\{\{c\}, \{b, c\}, \emptyset, \{b, c\}, \{c\}\} = \{b, c\}$

Hence:

- $C_3 = (X_3, Y_3) = (\{g1, g3, g5\}, \{b, c\})$

The Bordat algorithm

Direct subsumers of $C_2 = (X_2, Y_2) = (\{g3, g5\}, \{a, b, c, d\})$:

- $X_4 = X_2 \cup \{x \in G \setminus X_2 \ : \ f_{C_2}(x) = Y_4\}$
- $f_{C_2} : (G \setminus \{g3, g5\}) \times \{a, b, c, d\}$
- $Y_4 \in Max\{f_{C_2}(x) \ : \ x \in \{g1, g2, g4, g6\}\}$
- $Y_4 \in Max\{f_{C_2}(g1), f_{C_2}(g2), f_{C_2}(g4), f_{C_2}(g6)\} =$
  $Max\{\{b, c\}, \{a, c, d\}, \{a, d\}, \{a, c, d\}\} = \{\{b, c\}, \{a, c, d\}\}$

Hence:

- the first subsumer is identical to $C_3 = (X_3, Y_3) = (\{g1, g3, g5\}, \{b, c\})$
- the second subsumer $C_4 = (X_4, Y_4) = (\{g2, g3, g5, g6\}, \{a, c, d\})$

The Bordat algorithm

Direct subsumers of $C_3 = (X_3, Y_3) = (\{g1, g3, g5\}, \{b, c\})$:

- $X_5 = X_3 \cup \{x \in G \setminus X_3 \ : \ f_{C_3}(x) = Y_5\}$
- $f_{C_3} : (G \setminus \{g1, g3, g5\}) \times \{b, c\}$
- $Y_5 \in Max\{f_{C_3}(x) \ : \ x \in \{g2, g4, g6\}\}$
- $Y_5 \in Max\{f_{C_3}(g2), f_{C_3}(g4), f_{C_3}(g6)\} = Max\{\{c\}, \emptyset, \{c\}\} = \{c\}$

Hence:

- $C_5 = (X_5, Y_5) = (\{g1, g2, g3, g5, g6\}, \{c\})$

The Bordat algorithm

Direct subsumers of $C_4 = (X_4, Y_4) = (\{g2, g3, g5, g6\}, \{a, c, d\})$:

- $X_6 = X_4 \cup \{x \in G \setminus X_4 \ : \ f_{C_4}(x) = Y_6\}$
- $f_{C_4} : (G \setminus \{g2, g3, g5, g6\}) \times \{a, c, d\}$
- $Y_6 \in Max\{f_{C_4}(x) \ : \ x \in \{g1, g4\}$
- $Y_6 \in Max\{f_{C_4}(g1), f_{C_4}(g4)\}\} = Max\{\{c\}, \{a, d\}\}$

Hence:

- The first subsumer is identical to $C_5 = (\{g1, g2, g3, g5, g6\}, \{c\})$
- The second subsumer $C_6 = (\{g2, g3, g4, g5, g6\}, \{a, d\})$

The Bordat algorithm

Direct subsumers of $C_5 = (\{g1, g2, g3, g5, g6\}, \{c\})$:

- $X_7 = X_5 \cup \{x \in G \setminus X_5 \ : \ f_{C_5}(x) = Y_7\}$
- $f_{C_5} : (G \setminus \{g1, g2, g3, g5, g6\}) \times \{c\}$
- $Y_7 \in Max\{f_{C_5}(x) \ : \ x \in \{g4\}\} = Max\{f_{C_5}(g4)\}\} = \emptyset$

Hence:

- $C_7 = (\{g1, g2, g3, g4, g5, g6\}, \emptyset) = \top$

Similarly: the direct subsumer of $C_6 = (\{g2, g3, g4, g5, g6\}, \{a, d\})$ is C_7

The Bordat algorithm

Direct subsumers of $C_5 = (\{g1, g2, g3, g5, g6\}, \{c\})$:

- $X_7 = X_5 \cup \{x \in G \setminus X_5 \ : \ f_{C_5}(x) = Y_7\}$
- $f_{C_5} : (G \setminus \{g1, g2, g3, g5, g6\}) \times \{c\}$
- $Y_7 \in Max\{f_{C_5}(x) \ : \ x \in \{g4\}\} = Max\{f_{C_5}(g4)\}\} = \emptyset$

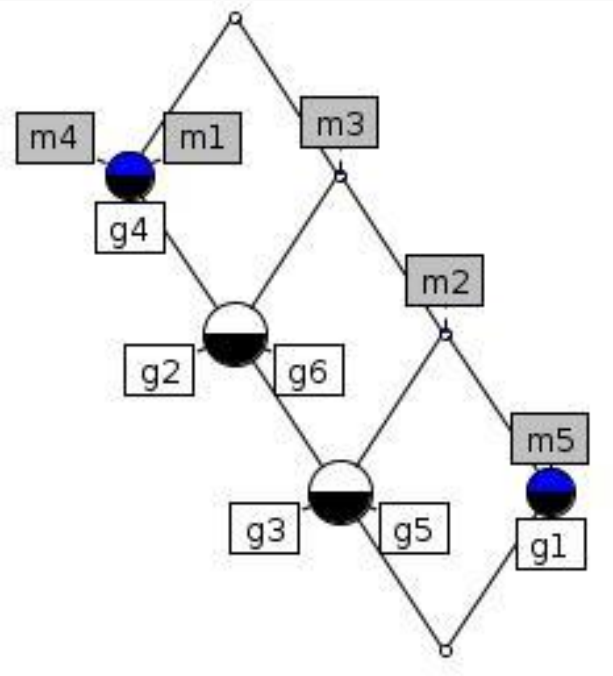
Hence:

- $C_7 = (\{g1, g2, g3, g4, g5, g6\}, \emptyset) = \top$

Similarly: the direct subsumer of $C_6 = (\{g2, g3, g4, g5, g6\}, \{a, d\})$ is C_7

The reduced labeling

G \ M	m1	m2	m3	m4	m5
g1		×	×		×
g2	×		×	×	
g3	×	×	×	×	
g4	×			×	
g5	×	×	×	×	
g6	×		×	×	



Reduced labeling: each object and each attribute appears only **once!**

Idea: attributes appear upwards (filters) **while** objects downwards (ideals).

Object and attribute concepts and reduced labelling...

The object concept of $g \in G$ is the concept $\gamma(g) = (\{g\}'', \{g\}')$. $\gamma(g)$ is the smallest concept containing g.

The attribute concept of $m \in M$ is the concept $\mu(m) = (\{m\}', \{m\}'')$. $\mu(m)$ is the biggest concept containing m.

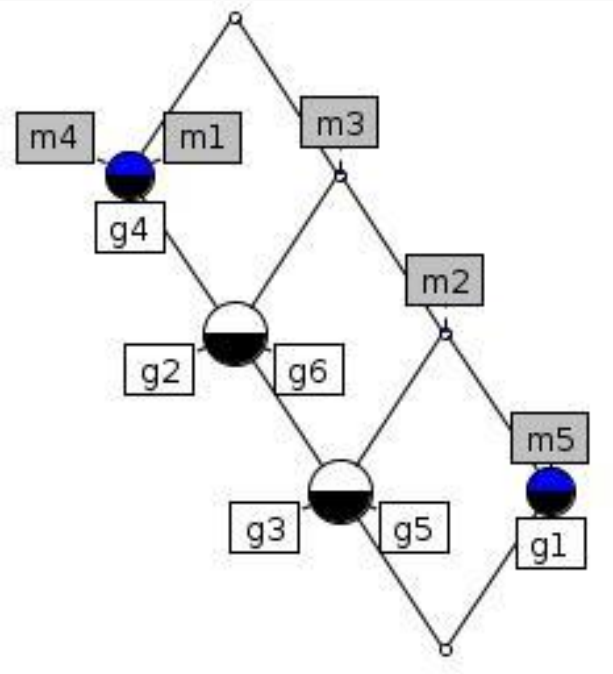
Reduced labeling: each object and attribute appears only once in the diagram.

The reduced labeling

G \ M	m1	m2	m3	m4	m5
g1		×	×		×
g2	×		×	×	
g3	×	×	×	×	
g4	×			×	
g5	×	×	×	×	
g6	×		×	×	

NB: For any concept (A, B),

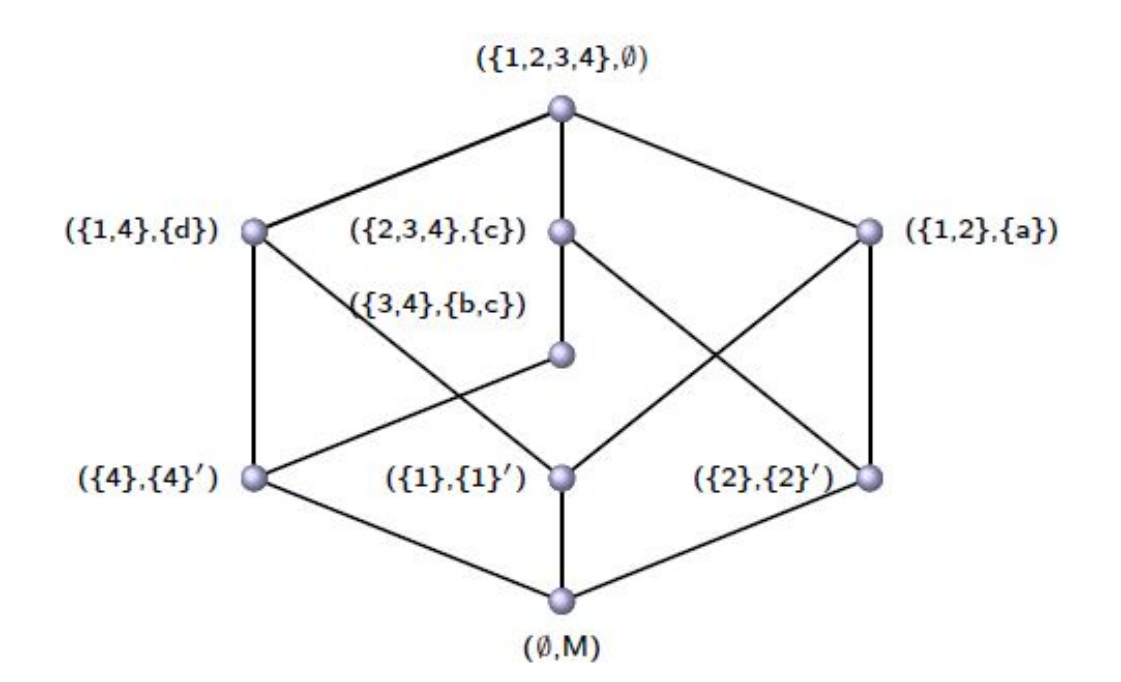
- $g \in A \iff \gamma(g) \leq (A, B)$
- $m \in B \iff (A, B) \leq \mu(m)$



NB: the attributes appear upwards (filters) and the objects downwards (ideals).

Exercise on 2D figures

	G \ M	a	b	c	d
1		×			×
2		×		×	
3			×	×	
4			×	×	×



From concept lattices to formal contexts

Let $(L, \leq)$ be a complete lattice. An element $x \in L$ is called

- $\vee$-irreducible if $x \neq \bigvee\{y \mid y < x\}$.
- $\wedge$-irreducible if $x \neq \bigwedge\{y \mid y > x\}$.

The sets of $\vee$- and $\wedge$-irreducibles of $(L, \leq)$ are denoted by $J(L)$ and $M(L)$, resp.

Corollary: For any lattice L, we have $L \cong \underline{\mathfrak{B}}(J(L), M(L), \leq)$.

Moreover: By interchanging G and M, we obtain the dual concept lattice $\underline{\mathfrak{B}}(G, M, I)^d \cong \underline{\mathfrak{B}}(M, G, I^{-1})$, through the anti-isomorphism $(B, A) \mapsto (A, B)$.

From concept lattices to formal contexts

Let $(L, \leq)$ be a complete lattice. An element $x \in L$ is called

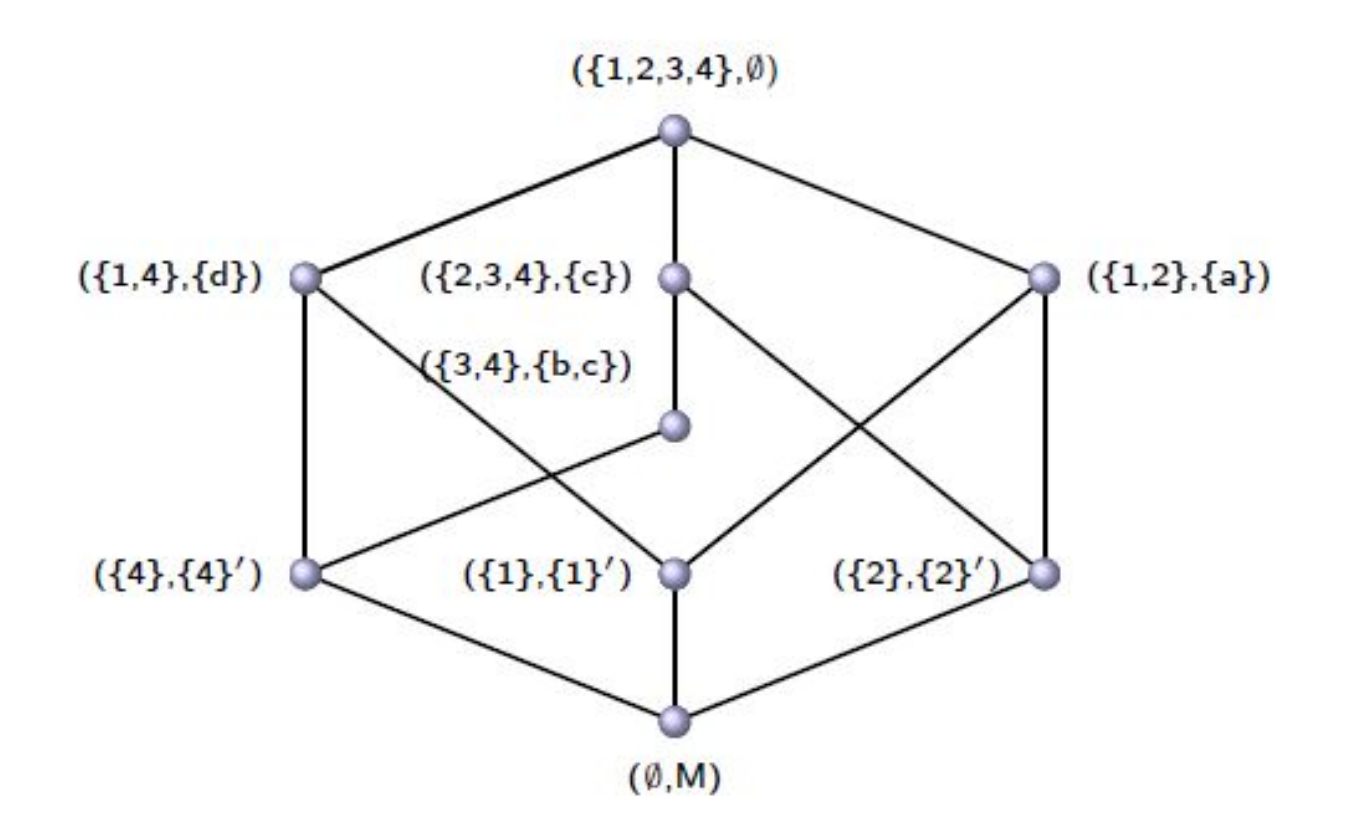
- $\vee$-irreducible if $x \neq \bigvee\{y \mid y < x\}$.
- $\wedge$-irreducible if $x \neq \bigwedge\{y \mid y > x\}$.

The sets of $\vee$- and $\wedge$-irreducibles of $(L, \leq)$ are denoted by $J(L)$ and $M(L)$, resp.

Corollary: For any lattice L, we have $L \cong \underline{\mathfrak{B}}(J(L), M(L), \leq)$.

Moreover: By interchanging G and M, we obtain the dual concept lattice $\underline{\mathfrak{B}}(G, M, I)^d \cong \underline{\mathfrak{B}}(M, G, I^{-1})$, through the anti-isomorphism $(B, A) \mapsto (A, B)$.

Back to the example...



	G \ M	a	b	c	d
1		×			×
2		×		×	
3			×	×	
4			×	×	×

a – has 3 vertices

b – has 4 vertices

c – has a right angle

d – equilateral

Reducing attribute sets

Let $\mathbb{K} = (G, M, I)$. Attribute $m \in M$ is **reducible** if $m' = G$ or

$$m' = \bigcap \{n' \mid n \in M \ \& \ n' \supset m'\}$$

NB: If m is reducible, **then**

$$\underline{\mathfrak{B}}(G, M, I) \cong \underline{\mathfrak{B}}(G, M \setminus \{m\}, I \cap (G \times (M \setminus \{m\})))$$

Example. Attribute m_k is reducible, since $m'_k = m'_i \cap m'_j$

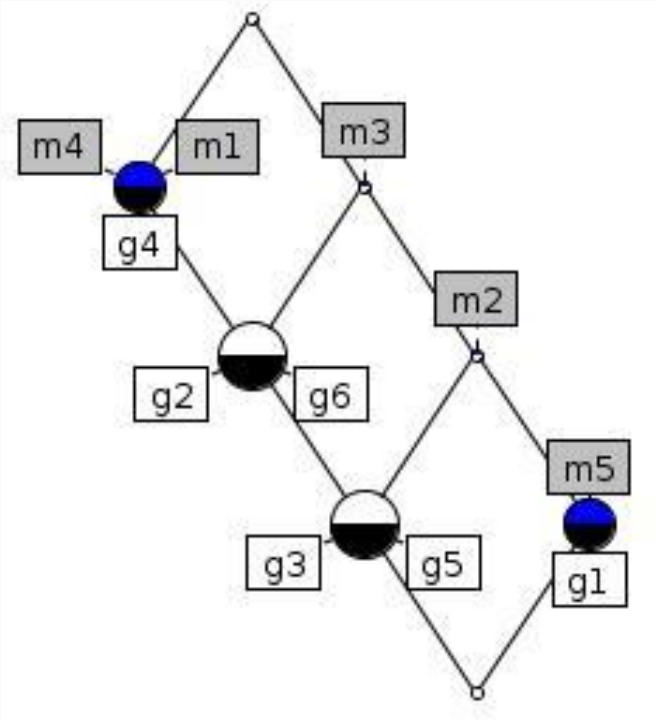
$G \setminus M$	$\dots$	m_i	$\dots$	m_j	$\dots$	m_k	$\dots$
g_1	$\dots$	$\times$	$\dots$		$\dots$		$\dots$
g_2	$\dots$	$\times$	$\dots$	$\times$	$\dots$	$\times$	$\dots$
g_3	$\dots$	$\times$	$\dots$	$\times$	$\dots$	$\times$	$\dots$
g_4	$\dots$		$\dots$	$\times$	$\dots$		$\dots$

Types of attributes

Introducing an attribute: an attribute α is introduced by a concept C when it is not present in any ascendant of C, i.e., the concept C corresponds to the attribute concept of α (the introducer of α).

Inheriting an attribute: an attribute α is inherited by a concept C when it is already present in a proper ascendant of C, i.e., C is subsumed by the introducer of α.

Types of attributes (examples)



$m3$ is introduced $(g12356, m3)$

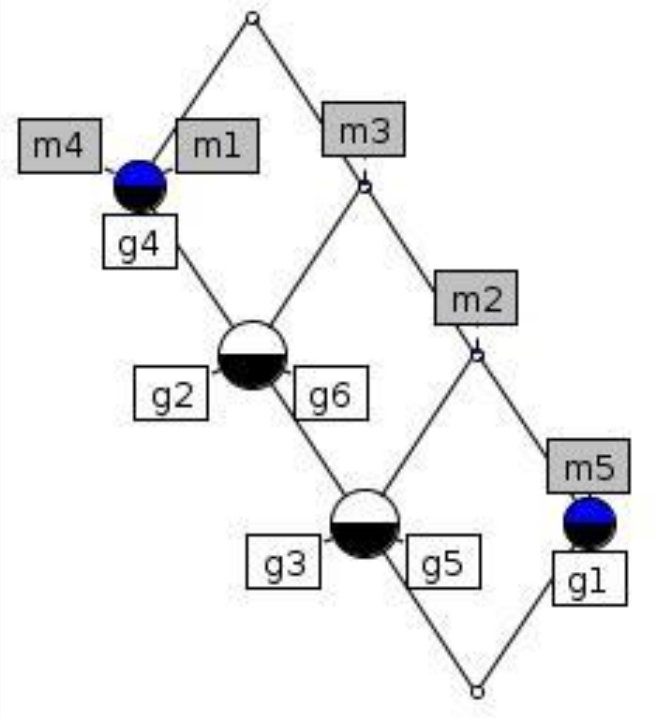
$m1$ and $m4$ are introduced by $(g23456, m14)$

$m2$ is introduced by $(g135, m23)$

$m3$ is inherited by $(g135, m23)$

$m1$, $m3$, and $m4$, are inherited by $(g2356, m134)$

Mutual implications between attributes having the same attribute-concept



Attributes having the same introducer are equivalent:

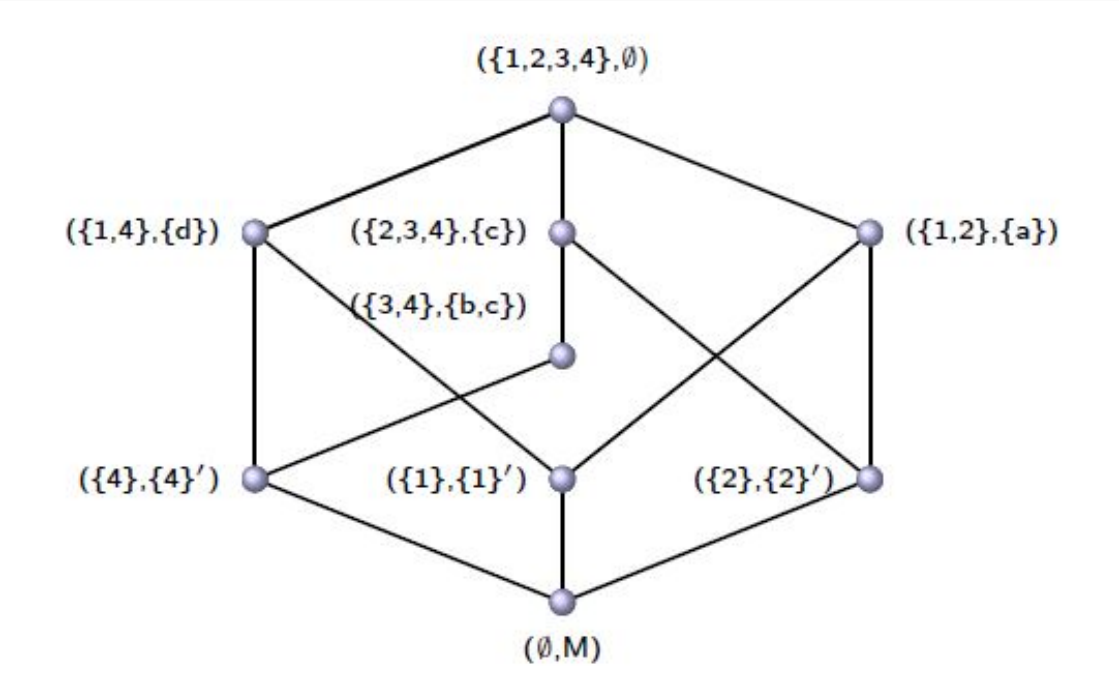
Ex: $m1 \longleftrightarrow m4$ for $(g23456, m14)$

Introduced attributes imply inherited attributes

If α is introduced, then it implies every inherited attribute in the introducer of α

Ex: $m2 \longrightarrow m3$ for $(g135, m23)$ and $m5 \longrightarrow m23$ for $(g1, m235)$

Exercise on 2D figures



	G \ M	a	b	c	d
1		×			×
2		×		×	
3			×	×	
4			×	×	×

a – has 3 vertices

b – has 4 vertices

c – has a right angle

d – equilateral

Exercise: What rules that can be extracted directly from the concept lattice.

Many-valued contexts

A formal context is the basic data type of FCA.

However data are often in form of a **many-valued context** (rel. database)

A many-valued context is a quadruple (G, M, W, J), where W is the set of **attribute values**, and the **incidence relation** $J \subseteq G \times M \times W$ is such that $(g, m, w) \in J$, and $(g, m, v) \in J$ **implies** $w = v$.

G \ M	m1	m2	m3
g1	1	3	4
g2	2	2	3
g3	4	1	1
g4	2	2	1

An attribute m is complete if $\forall g \in G \exists w \in W$ such that $(g, m, w) \in J$

A many-valued context is complete if all its attributes are complete.

NB: For complete many-valued contexts the value of attribute m for object g is denoted by $m(g)$, i.e., $(g, m, m(g)) \in J$.

Many-valued contexts

A formal context is the basic data type of FCA.

However data are often in form of a **many-valued context** (rel. database)

A many-valued context is a quadruple (G, M, W, J), where W is the set of **attribute values**, and the **incidence relation** $J \subseteq G \times M \times W$ is such that $(g, m, w) \in J$, and $(g, m, v) \in J$ **implies** $w = v$.

G \ M	m1	m2	m3
g1	1	3	4
g2	2	2	3
g3	4	1	1
g4	2	2	1

An attribute m is complete if $\forall g \in G \exists w \in W$ such that $(g, m, w) \in J$

A many-valued context is complete if all its attributes are complete.

NB: For complete many-valued contexts the value of attribute m for object g is denoted by $m(g)$, i.e., $(g, m, m(g)) \in J$.

Conceptual scaling

Many-valued contexts are translated a usual context via conceptual scaling. This is not automatic and some choices have to be made.

Examples of scalings:

- Nominal: $K = (N, N, =)$
- Ordinal: $K = (N, N, \leq)$
- Interordinal: $K = (N, N, \leq \cup \geq)$

A numerical example

G \ M	m1	m2	m3
g1	1	3	4
g2	2	2	3
g3	4	1	1
g4	2	2	1

Nominal Scaling:

G \ M	m1=1	m1=2	m1=4	m2=1	m2=2	m2=3	m3=1	m3=3	m3=4
g1	×					×			×
g2		×			×			×	
g3			×	×			×		
g4		×			×		×		

Interordinal Scaling:

G \ M	m1≤1	m1≥1	m1≤2	m1≥2	m1≤3	m1≥3	m1≤4	m1≥4	m2≤1	m2≥1
g1	×	×	×		×		×			×
g2		×	×	×	×		×			×
g3		×		×		×	×	×	×	×
g4		×	×	×	×		×			×

A refined algorithm to construct concept lattices:
`AllClosure` and `NextClosure`

The lectic order

Consider a linear ordering $<$ of M say $\quad M = \{a_1 < a_2 < \cdots < a_k\}$

Lectic order: $A \subseteq M$ is lectically smaller than $B \subseteq M$, $A < B$, if there is $i \in M$ such that:

$$i \in B \setminus A \;\; \textbf{and} \;\; \forall j < i \;\; \text{we have} \;\; j \in A \Leftrightarrow j \in B$$

Refinement: $A <_i B$ **iff** $i \in B \setminus A$ **and** $\forall j < i$ we have $j \in A \Leftrightarrow j \in B$

NB1: Take the characteristic vectors and consider the reversed lexic. order

NB2: If $A \subseteq B$, then $A < B$ (take the first in $B \setminus A$)

Example: Let $M = \{a < b < c < d < e < f\}$. Then $\{a, c, e\} < \{a, c, d\}$ **and**

$$\{a, c, e\} <_d \{a, c, d\} \;\; \textbf{but} \;\; \{a, c, e\} \not<_e \{a, c, d\}$$

The lectic order

Consider a linear ordering $<$ of M say $\quad M = \{a_1 < a_2 < \cdots < a_k\}$

Lectic order: $A \subseteq M$ is lectically smaller than $B \subseteq M$, $A < B$, if there is $i \in M$ such that:

$$i \in B \setminus A \;\; \textbf{and} \;\; \forall j < i \;\; \text{we have} \;\; j \in A \Leftrightarrow j \in B$$

Refinement: $A <_i B$ **iff** $i \in B \setminus A$ **and** $\forall j < i$ we have $j \in A \Leftrightarrow j \in B$

NB1: Take the characteristic vectors and consider the reversed lexic. order

NB2: If $A \subseteq B$, then $A < B$ (take the first in $B \setminus A$)

Example: Let $M = \{a < b < c < d < e < f\}$. Then $\{a, c, e\} < \{a, c, d\}$ **and**

$$\{a, c, e\} <_d \{a, c, d\} \;\; \textbf{but} \; \{a, c, e\} \not<_e \{a, c, d\}$$

Closures in lectic order

NB3: For $A, B \subseteq M$, $A < B$ **iff** $A <_i B$, for some $i \in M$

NB4: For $A, B, C \subseteq M$, If $A <_i B$ **and** $A <_j C$ for $i < j$, **then** $C <_i B$

Let $A \subseteq M$ and $i \in M$. **Define**

$$A + i = ((A \cap \{j : j < i\}) \cup \{i\}) \quad \textbf{and} \quad A \oplus i = (A + i)''$$

Example: Let $A = \{a, c, d, f\}$ and $i = e$. **Then**

$$A \oplus e = ((\{a, c, d, f\} \cap \{a, b, c, d\}) \cup \{e\})'' = (\{a, c, d, e\})''$$

Theorem: The smallest concept intent larger than a given set $A \subset M$ with respect to the lectic order is $A \oplus i = (A + i)''$, with i being the largest element of M with $A <_i A \oplus i$

Closures in lectic order

NB3: For $A, B \subseteq M$, $A < B$ **iff** $A <_i B$, for some $i \in M$

NB4: For $A, B, C \subseteq M$, If $A <_i B$ **and** $A <_j C$ for $i < j$, **then** $C <_i B$

Let $A \subseteq M$ and $i \in M$. **Define**

$$A + i = ((A \cap \{j : j < i\}) \cup \{i\}) \quad \textbf{and} \quad A \oplus i = (A + i)''$$

Example: Let $A = \{a, c, d, f\}$ and $i = e$. **Then**

$$A \oplus e = ((\{a, c, d, f\} \cap \{a, b, c, d\}) \cup \{e\})'' = (\{a, c, d, e\})''$$

Theorem: The smallest concept intent larger than a given set $A \subset M$ with respect to the lectic order is $A \oplus i = (A + i)''$, with i being the largest element of M with $A <_i A \oplus i$

Closures in lectic order

NB3: For $A, B \subseteq M$, $A < B$ **iff** $A <_i B$, for some $i \in M$

NB4: For $A, B, C \subseteq M$, If $A <_i B$ **and** $A <_j C$ for $i < j$, **then** $C <_i B$

Let $A \subseteq M$ and $i \in M$. **Define**

$$A + i = ((A \cap \{j : j < i\}) \cup \{i\}) \quad \textbf{and} \quad A \oplus i = (A + i)''$$

Example: Let $A = \{a, c, d, f\}$ and $i = e$. **Then**

$$A \oplus e = ((\{a, c, d, f\} \cap \{a, b, c, d\}) \cup \{e\})'' = (\{a, c, d, e\})''$$

Theorem: The smallest concept intent larger than a given set $A \subset M$ with respect to the lectic order is $A \oplus i = (A + i)''$, with i being the largest element of M with $A <_i A \oplus i$

Algorithms `AllClosure` and `NextClosure`

`AllClosure`: Generates the partially ordered set of closed attribute sets

Input: Closure operator $X \mapsto X''$ (formal context) on a set of attributes M

Output: All closed attribute sets in lectic order

1. $A \leftarrow \varnothing''$

2. While $A \neq M$ **do**

2.1. Output: A

2.2. $A \leftarrow \texttt{NextClosure}(A, M, '')$

Algorithm `NextClosure` (cont.)

`NextClosure`: Generates a the lectically next closed attribute set

Input: Closure operator $X \mapsto X''$ on a set of attributes M and $A \subseteq M$

Output: The lectically next closed attribute set, or M if it doesn't exist

1. For all $i \in M$ (in descending order) **do**

1.1. if $A <_i A \oplus i$ **then return** $A \oplus i$

2. return M

Exercise: compute the concept lattice of our example

	G \ M	a	b	c	d
1		×			×
2		×		×	
3			×	×	
4			×	×	×

Objects:

- **1** – equilateral triangle
- **2** – rectangle triangle
- **3** – rectangle
- **4** – square

Attributes:

- **a** – has 3 vertices
- **b** – has 4 vertices
- **c** – has a direct angle
- **d** – equilateral

Exercise: compute the concept lattice of our example

	G \ M	a	b	c	d
1		×			×
2		×		×	
3			×	×	
4			×	×	×

Tip: use a table for the computation!

A	i	$A + i$	$A \oplus i$	$A <_i A \oplus i$?	New intent?
$\varnothing$	d	$\{d\}$	$\{d\}'' = \{1, 4\}' = \{d\}$	yes	$\{d\}$
$\{d\}$	d	$\{d\}$	$\{d\}$	no	–
$\{d\}$	c	$\{c\}$	$\{c\}$	yes	$\{c\}$
$\{c\}$	d	$\{c, d\}$	$\{c, d\}'' = \{4\}' = \{b, c, d\}$	no	–
$\{c\}$	c	$\{c\}$	$\{c\}$	no	–
$\{c\}$	b	$\{b\}$	$\{b, c\}$	yes	$\{b, c\}$
$\{b, c\}$	d	$\{b, c, d\}$	$\{b, c, d\}$	yes	$\{b, c, d\}$
$\{b, c, d\}$	d	$\{b, c, d\}$	$\{b, c, d\}$	no	–
$\{b, c, d\}$	c	$\{b, c\}$	$\{b, c\}$	no	–
$\{b, c, d\}$	b	$\{b\}$	$\{b, c\}$	no	–
$\{b, c, d\}$	a	$\{a\}$	$\{a\}$	yes	$\{a\}$
$\{a\}$	d	$\{a, d\}$	$\{a, d\}$	yes	$\{a, d\}$
$\{a, d\}$	d	$\{a, d\}$	$\{a, d\}$	no	–
$\{a, d\}$	c	$\{a, c\}$	$\{a, c\}$	yes	$\{a, c\}$
$\{a, c\}$	d	$\{a, c, d\}$	$\{a, b, c, d\}$	no	–
$\{a, c\}$	c	$\{a, c\}$	$\{a, c\}$	no	–
$\{a, c\}$	b	$\{a, b\}$	$\{a, b, c, d\}$	yes	$\{a, b, c, d\}$

Some complexity issues

NB1: Consider context $\mathbb{K} = (G, G \neq)$ for an arbitrary finite set G. Then $\underline{\mathfrak{B}}(\mathbb{K})$ is isomorphic to the Boolean lattice 2^G with $2^{|G|}$ concepts.

NB2: Given a context $\mathbb{K} = (G, M, I)$, computing the size of $\underline{\mathfrak{B}}(\mathbb{K})$ is #P-complete [Kuznetsov 1989, 2001].

Intuition: #P is the class of counting problems associated with the decision problems in NP.

More formally: A problem is in #P if there is a non-deterministic, polynomial time Turing machine (TM) that, for each instance I of the problem, has a number of accepting computations that is exactly equal to the number of distinct solutions for instance I.

#P-complete: #P and #P-**hard**, i.e., any problem in #P can be reduced to it by a TM (or a #P-complete problem can be reduced to it).

NB: #P = P $\Rightarrow$ NP = P.

Some complexity issues

NB1: Consider context $\mathbb{K} = (G, G \neq)$ for an arbitrary finite set G. Then $\underline{\mathfrak{B}}(\mathbb{K})$ is isomorphic to the Boolean lattice 2^G with $2^{|G|}$ concepts.

NB2: Given a context $\mathbb{K} = (G, M, I)$, computing the size of $\underline{\mathfrak{B}}(\mathbb{K})$ is #P-complete [Kuznetsov 1989, 2001].

Intuition: #P is the class of counting problems associated with the decision problems in NP.

More formally: A problem is in #P if there is a non-deterministic, polynomial time Turing machine (TM) that, for each instance I of the problem, has a number of accepting computations that is exactly equal to the number of distinct solutions for instance I.

#P-complete: #P and #P-**hard**, i.e., any problem in #P can be reduced to it by a TM (or a #P-complete problem can be reduced to it).

NB: $\#\mathrm{P} = \mathrm{P} \Rightarrow \mathrm{NP} = \mathrm{P}$.

Some complexity mitigation

Good news: Computing concepts is possible with polynomial delay

Polynomial delay: If the time between the output of one solution and the next is bounded by a polynomial function of the input size (in the worst case)

Idea 1: Compute only concepts with big extents, i.e., large support (it can be done with polynomial delay)

Idea 2: Compute concepts with high stability or other quality indices.

Some complexity mitigation

Good news: Computing concepts is possible with polynomial delay

Polynomial delay: If the time between the output of one solution and the next is bounded by a polynomial function of the input size (in the worst case)

Idea 1: Compute only concepts with big extents, i.e., large support (it can be done with polynomial delay)

Idea 2: Compute concepts with high stability or other quality indices.

References

- Introduction to Formal Concept Analysis. R. Belohlavek, Palacky University, Olomouc
- C. Carpineto, G. Romano. Concept Data Analysis: Theory and Applications, John Wiley Sons, 2004.
- Finite Ordered Sets: Concepts, Results and Uses. N. Caspard, B. Leclerc, B. Monjardet, Cambridge University Press, 2012.
- Bernhard Ganter and Rudolf Wille, Formal Concept Analysis, Springer, 1999.
- Formal Concept Analysis, Foundations and Applications. B. Ganter, G. Stumme, R. Wille editors, Lecture Notes in Computer Science 3626, Springer, 2005.
- S.O. Kuznetsov, *Mathematical aspects of concept analysis.* J.Math. Sci., vol. 80(2), pp.1654-1698 (1996).
- S.O. Kuznetsov, *On Computing the Size of a Lattice and Related Decision Problems.* Order, vol. 18, no. 4, pp. 313-321 (2001).
- S.O. Kuznetsov, *On the Intractability of Computing the Duquenne-Guigues Base.* J. Univ. Comp. Sci., vol. 10, no. 8, pp. 927-933 (2004).

Exercises

Compute the concept lattices for each of the following formal contexts

1.

Objects/Attributes	1	2	3
A	X	X	X
B	X		X
C		X	X
D	X		

2.

model/functionality	**Mobile**	**Phone**	**Fax**	**Paper fax**
Sinus 14		X		
Nokia 6110	X	X		
T-Fax 301			X	X
T-Fax 360 PC				X

Implications & functional dependencies

Implications from concept lattices and implication models

Motivation: Mine attribute dependencies in cross-tables, i.e., implications!

Examples: implications represent data dependencies such as

- every number divisible by 2 and 3 is divisible by 6, or
- every patient with symptoms s_2 and s_5 also has symptoms s_1 and s_3.

Implication $A \to B$ holds in (G, M, I) if $A' \subseteq B'$, i.e., each object having all attributes from A also has all attributes from B.

Recall: For $A \to B$, $\bigwedge_{a \in A} \mu(a) \sqsubseteq \bigwedge_{b \in B} \mu(b)$.

Homework: frequent itemsets (itemset whose support is greater than a given threshold), `APriori` (mines implications based on frequent itemsets), ...

Def.: $T \subseteq M$ is a **model** of $A \to B$, denoted $T \models A \to B$, if $A \not\subseteq T$ or $B \subseteq T$.
If $\mathcal{I}$ set of implications, then $T \models \mathcal{I}$ if $T \models A \to B$, for every $A \to B \in \mathcal{I}$.
We denote the set of models of $\mathcal{I}$ by $Mod(\mathcal{I})$

Implications from concept lattices and implication models

Motivation: Mine attribute dependencies in cross-tables, i.e., implications!

Examples: implications represent data dependencies such as

- every number divisible by 2 and 3 is divisible by 6, or
- every patient with symptoms s_2 and s_5 also has symptoms s_1 and s_3.

Implication $A \to B$ holds in (G, M, I) if $A' \subseteq B'$, i.e., each object having all attributes from A also has all attributes from B.

Recall: For $A \to B$, $\bigwedge_{a \in A} \mu(a) \sqsubseteq \bigwedge_{b \in B} \mu(b)$.

Homework: frequent itemsets (itemset whose support is greater than a given threshold), `APriori` (mines implications based on frequent itemsets), ...

Def.: $T \subseteq M$ is a **model** of $A \to B$, denoted $T \models A \to B$, if $A \not\subseteq T$ or $B \subseteq T$.
If $\mathcal{I}$ set of implications, then $T \models \mathcal{I}$ if $T \models A \to B$, for every $A \to B \in \mathcal{I}$.
We denote the set of models of $\mathcal{I}$ by $Mod(\mathcal{I})$

Implications from concept lattices and implication models

Motivation: Mine attribute dependencies in cross-tables, i.e., implications!

Examples: implications represent data dependencies such as

- every number divisible by 2 and 3 is divisible by 6, or
- every patient with symptoms s_2 and s_5 also has symptoms s_1 and s_3.

Implication $A \to B$ holds in (G, M, I) if $A' \subseteq B'$, i.e., each object having all attributes from A also has all attributes from B.

Recall: For $A \to B$, $\bigwedge_{a \in A} \mu(a) \sqsubseteq \bigwedge_{b \in B} \mu(b)$.

Homework: frequent itemsets (itemset whose support is greater than a given threshold), `APriori` (mines implications based on frequent itemsets), ...

Def.: $T \subseteq M$ is a **model** of $A \to B$, denoted $T \models A \to B$, if $A \not\subseteq T$ or $B \subseteq T$.
If $\mathcal{I}$ set of implications, then $T \models \mathcal{I}$ if $T \models A \to B$, for every $A \to B \in \mathcal{I}$.
We denote the set of models of $\mathcal{I}$ by $Mod(\mathcal{I})$

A basic algorithm to compute implication closures

Problem: For $X \subset M$, determine the $\mathcal{I}$-closure of X:

$$\mathcal{I}(X) = \bigcap \{B \colon X \subseteq B \subseteq M,\ B \models \mathcal{I}\}$$

NB: $\mathcal{I}(X)$ is the smallest model of $\mathcal{I}$ that contains X.

For $X \subset M$ and an implication set $\mathcal{I}$, **Define:**

$$X^{\mathcal{I}} = X \cup \bigcup \{B \colon A \to B \in \mathcal{I}, A \subseteq X\}$$

Result: $\mathcal{I}(X) = (((X^{\mathcal{I}})^{\mathcal{I}})^{\mathcal{I}})^{\cdots}$ and $\mathcal{I}(X)^{\mathcal{I}} = \mathcal{I}(X)$

`ImpClosure`: Computes the $\mathcal{I}$-closure of $X \subseteq M$ (repeated until $X^{\mathcal{I}} = X$)

Repeat: For all $A \to B \in \mathcal{I}$ **do**

  if $A \subseteq X$ **then** $X \leftarrow X \cup B, \quad \mathcal{I} \leftarrow \mathcal{I} \setminus \{A \to B\}$

Return: X

However: Quadratic in the number of implications...

A basic algorithm to compute implication closures

Problem: For $X \subset M$, determine the $\mathcal{I}$-closure of X:

$$\mathcal{I}(X) = \bigcap \{B \colon X \subseteq B \subseteq M,\ B \models \mathcal{I}\}$$

NB: $\mathcal{I}(X)$ is the smallest model of $\mathcal{I}$ that contains X.

For $X \subset M$ and an implication set $\mathcal{I}$, **Define:**

$$X^{\mathcal{I}} = X \cup \bigcup \{B \colon A \to B \in \mathcal{I}, A \subseteq X\}$$

Result: $\mathcal{I}(X) = (((X^{\mathcal{I}})^{\mathcal{I}})^{\mathcal{I}})^{\cdots}$ and $\mathcal{I}(X)^{\mathcal{I}} = \mathcal{I}(X)$

`ImpClosure`**:** Computes the $\mathcal{I}$-closure of $X \subseteq M$ (repeated until $X^{\mathcal{I}} = X$)

Repeat: For all $A \to B \in \mathcal{I}$ **do**

  if $A \subseteq X$ **then** $X \leftarrow X \cup B, \quad \mathcal{I} \leftarrow \mathcal{I} \setminus \{A \to B\}$

Return: X

However: Quadratic in the number of implications...

A basic algorithm to compute implication closures

Problem: For $X \subset M$, determine the $\mathcal{I}$-closure of X:

$$\mathcal{I}(X) = \bigcap \{B \colon X \subseteq B \subseteq M,\ B \models \mathcal{I}\}$$

NB: $\mathcal{I}(X)$ is the smallest model of $\mathcal{I}$ that contains X.

For $X \subset M$ and an implication set $\mathcal{I}$, **Define:**

$$X^{\mathcal{I}} = X \cup \bigcup \{B \colon A \to B \in \mathcal{I}, A \subseteq X\}$$

Result: $\mathcal{I}(X) = (((X^{\mathcal{I}})^{\mathcal{I}})^{\mathcal{I}})^{\cdots}$ and $\mathcal{I}(X)^{\mathcal{I}} = \mathcal{I}(X)$

`ImpClosure`**:** Computes the $\mathcal{I}$-closure of $X \subseteq M$ (repeated until $X^{\mathcal{I}} = X$)

Repeat: For all $A \to B \in \mathcal{I}$ **do**

if $A \subseteq X$ **then** $X \leftarrow X \cup B, \quad \mathcal{I} \leftarrow \mathcal{I} \setminus \{A \to B\}$

Return: X

However: Quadratic in the number of implications...

Implications from a logical perspective: bases

Implications satisfy **Armstrong rules**:

$$\frac{}{X \to X} \quad , \quad \frac{X \to Y}{X \cup Z \to Y} \quad , \quad \frac{X \to Y, Y \cup Z \to W}{X \cup Z \to W},$$

An **implication cover** is a subset of implications from which all other implications can be derived by means of Armstrong rules.

An **implication base** is a minimal (by inclusion) implication cover.

Recall: $\mathcal{I} \models A \to B$ (semantic), $\mathcal{I} \vdash A \to B$ (syntactic), and coincide!

Implications from a logical perspective: bases

Implications satisfy **Armstrong rules**:

$$\frac{}{X \to X} \quad , \quad \frac{X \to Y}{X \cup Z \to Y} \quad , \quad \frac{X \to Y, Y \cup Z \to W}{X \cup Z \to W},$$

An **implication cover** is a subset of implications from which all other implications can be derived by means of Armstrong rules.

An **implication base** is a minimal (by inclusion) implication cover.

Recall: $\mathcal{I} \models A \to B$ (semantic), $\mathcal{I} \vdash A \to B$ (syntactic), and coincide!

Exercises I

Consider the following attributes: normal blood pressure (nbp), high blood pressure (hbp), watches TV (TV), eats unhealthy food (uf), runs regularly (r).

-	**nbp**	**hbp**	**TV**	**uf**	**r**
A	X				X
B	X			X	X
C		X	X	X	
D		X		X	
E	X				

Check whether the following implications are valid: $\{r\} \rightarrow \{nbp\}$, $\{TV, uf\} \rightarrow \{hbp\}$, $\{TV\} \rightarrow \{hbp\}$, $\{uf\} \rightarrow \{hbp\}$, $\{nbp\} \rightarrow \{r\}$, $\{nbp,hbp\} \rightarrow \{r,TV\}$, $\{uf,r\} \rightarrow \{r\}$

Exercises II

Consider the following theories:

- $T_1 = \{\{y_3\} \to \{y_1, y_2\}, \{y_1, y_3\} \to \{y_2\}\}$.
- $T_2 = \{\{y_3\} \to \{y_1, y_2\}\}$.
- $T_3 = \{\{y_1, y_3\} \to \{y_2\}\}$.
- $T_4 = \{\{y1\} \to \{y_3\}, \{y_3\} \to \{y_1\}, \{y_2\} \to \{y_2\}\}$.
- $T_5 = \varnothing$.
- $T_6 = \{\varnothing \to \{y_1\}, \varnothing \to \{y_3\}\}$.
- $T_7 = \{\{y_1\} \to \varnothing, \{y_2\} \to \varnothing, \{y3\} \to \varnothing\}$.
- $T_8 = \{\{y_1\} \to \{y_2\}, \{y_2\} \to \{y_3\}, \{y_3\} \to \{y_1\}\}$.

Compute the corresponding sets of models.

Exercises III

1. Consider the following theories T together with an implication $A \to B$ (over $M = \{y_1, y_2, y_3\}$). Decide whether $T \models A \to B$
   - $T = \{\{y_3\} \to \{y_1, y_2\}, \{y_1, y_3\} \to \{y_2\}\}$, where $A \to B$ is $\{y_2, y_3\} \to \{y_1\}$.
   - $T = \{\{y_3\} \to \{y_1, y_2\}, \{y_1, y_3\} \to \{y_2\}\}$, where $A \to B$ is $\{y_2\} \to \{y_1\}$.

2. Let $T = \{\{y_1, y_4\} \to \{y_3\}, \{y_2, y_4\} \to \{y_1\}, \{y_1, y_2\} \to \{y_4, y_7\}, \{y_2, y_7\} \to \{y_3\}, \{y_6\} \to \{y_4\}, \{y_2, y_8\} \to \{y_3\}, \{y_9\} \to \{y_1, y_2, y_7\}\}$ (over $M = \{y_1, \ldots, y_{10}\}$). Decide whether $T \models A \to B$ when $A \to B$ is
   - $\{y_2, y_5, y_6\} \to \{y_3, y_7\}$.
   - $\{y_1, y_2, y_8\} \to \{y_4, y_7\}$.

Non-redundant bases of implications

A set $\mathcal{I}$ of implications is called non-redundant if for any $A \to B \in \mathcal{I}$,

$$\mathcal{I} \setminus \{A \to B\} \not\models A \to B.$$

To remove redundancy we can use:

Repeat: For all $A \to B \in \mathcal{I}$ **do**

$\mathcal{I}' = \mathcal{I} \setminus \{A \to B\}$
if $\mathcal{I}' \models A \to B$ **then** $\mathcal{I} = \mathcal{I}'$

Return: $\mathcal{I}$

NB1: An **(implication) basis** is a non-redundant cover (complete)

NB2: $\mathcal{I}$ is a cover of (G, M, I) iff $Mod(\mathcal{I})$ are exactly the intents of $\underline{\mathfrak{B}}(G, M, I)$

NB: Any basis $\mathcal{I}$ can be seen as a "partial" closure operator!

Question: How to compute X'', for any $X \subseteq M$?

Non-redundant bases of implications

A set $\mathcal{I}$ of implications is called non-redundant if for any $A \to B \in \mathcal{I}$,

$$\mathcal{I} \setminus \{A \to B\} \not\models A \to B.$$

To remove redundancy we can use:

Repeat: For all $A \to B \in \mathcal{I}$ **do**

$\mathcal{I}' = \mathcal{I} \setminus \{A \to B\}$
if $\mathcal{I}' \models A \to B$ **then** $\mathcal{I} = \mathcal{I}'$

Return: $\mathcal{I}$

NB1: An **(implication) basis** is a non-redundant cover (complete)

NB2: $\mathcal{I}$ is a cover of (G, M, I) iff $Mod(\mathcal{I})$ are exactly the intents of $\underline{\mathfrak{B}}(G, M, I)$

NB: Any basis $\mathcal{I}$ can be seen as a "partial" closure operator!

Question: How to compute X'', for any $X \subseteq M$?

Non-redundant bases of implications

A set $\mathcal{I}$ of implications is called non-redundant if for any $A \to B \in \mathcal{I}$,

$$\mathcal{I} \setminus \{A \to B\} \not\models A \to B.$$

To remove redundancy we can use:

Repeat: For all $A \to B \in \mathcal{I}$ **do**

$\mathcal{I}' = \mathcal{I} \setminus \{A \to B\}$
if $\mathcal{I}' \models A \to B$ **then** $\mathcal{I} = \mathcal{I}'$

Return: $\mathcal{I}$

NB1: An **(implication) basis** is a non-redundant cover (complete)

NB2: $\mathcal{I}$ is a cover of (G, M, I) iff $Mod(\mathcal{I})$ are exactly the intents of $\underline{\mathfrak{B}}(G, M, I)$

NB: Any basis $\mathcal{I}$ can be seen as a "partial" closure operator!

Question: How to compute X'', for any $X \subseteq M$?

Generator implication cover

$D \subseteq M$ is a **generator** of a closed $B = B'' \subseteq M$, if $D \subseteq B$ and $D'' = B$.

$D \subseteq M$ is a **minimal generator** (key) if for all $E \subset D$, $\;E'' \neq D'' = B''$.

It is called **nontrivial** if $D \neq D'' = B''$. The set of all nontrivial minimal generators of B is denoted by $\text{mingen}(B)$.

Generator implication cover: $\{F \to (F'' \setminus F) \mid F \subseteq M,\, F \in \text{mingen}(F'')\}$.

Direct (proper premise) basis

$A \subseteq M$ is a **proper premise** if

$$A'' \neq A \cup \bigcup_{n \in A} (A \setminus \{n\})''.$$

NB1: The closure of a proper premise cannot be deduced from the union of closures of proper subsets of A.

NB2: Every proper premise is a minimal generator, but not vice versa.

Direct basis[1]**:** $\mathcal{D} = \{A \to (A'' \setminus A) \mid A \text{ is a proper premise}\}$

Or: $\mathcal{D} = \{A \to A'' \mid A \text{ is a proper premise}\}$

[1] Also: "Proper premise base", "Direct Canonical Basis", "Karell Basis",
from K. Bertet, B. Monjardet. The multiple facets of the canonical direct unit implicational basis, Theoretical Computer Science, Vol. 411, Issues 22-24, 2010, Pages 2155-2166, ISSN 0304-3975, https://doi.org/10.1016/j.tcs.2009.12.021.

Minimum implication (canonical) basis

$P \subseteq M$ is a **pseudo-intent** (or **pseudo-closed**) if it is not an intent ($P \neq P''$), and for any pseudo-intent $Q \subset P$, we have $Q'' \subset P$.

Duquenne-Guigues (canonical) basis:

$$\mathcal{I} = \{P \to (P'' \setminus P) \mid P \text{ pseudo-intent } \}.$$

Or: $\mathcal{I} = \{P \to P'' \mid P \text{ pseudo-intent}\}$.

Theorem: The D-G basis is a (cardinality) minimal implication basis.

Nevertheless: It can have exponential size!

Minimum implication (canonical) basis

$P \subseteq M$ is a **pseudo-intent** (or **pseudo-closed**) if it is not an intent ($P \neq P''$), and for any pseudo-intent $Q \subset P$, we have $Q'' \subset P$.

Duquenne-Guigues (canonical) basis:

$$\mathcal{I} = \{P \to (P'' \setminus P) \mid P \text{ pseudo-intent } \}.$$

Or: $\mathcal{I} = \{P \to P'' \mid P \text{ pseudo-intent}\}$.

Theorem: The D-G basis is a (cardinality) minimal implication basis.

Nevertheless: It can have exponential size!

Exercises IV

Recall Exercise I: normal blood pressure (nbp), high blood pressure (hbp), watches TV (TV), eats unhealthy food (uf), runs regularly (r).

-	**nbp**	**hbp**	**TV**	**uf**	**r**
A	X				X
B	X			X	X
C		X	X	X	
D		X		X	
E	X				

Check whether the following theories are complete:

1. $T = \{\{\text{r}\} \to \{\text{nbp}\}, \{\text{TV, uf}\} \to \{\text{hbp}\}, \{\text{r,uf}\} \to \{\text{TV}\}$.
2. $T = \{\{\text{r}\} \to \{\text{nbp}\}, \{\text{TV, uf}\} \to \{\text{hbp}\}, \{\text{TV}\} \to \{\text{uf}\}, \{\text{TV}\} \to \{\text{hbp}\}\}$.
3. $T = \{\{\text{r}\} \to \{\text{nbp}\}, \{\text{TV, uf}\} \to \{\text{hbp}\}, \{\text{TV}\} \to \{\text{uf}\},$
   $\{\text{TV}\} \to \{\text{hbp}\}, \{\text{hbp,TV}\} \to \{\text{uf}\}, \{\text{nbp,uf}\} \to \{\text{r}\}, \{\text{hbp}\} \to \{\text{uf}\},$
   $\{\text{uf,r}\} \to \{\text{nbp}\}, \{\text{nbp,TV}\} \to \{\text{r}\}, \{\text{hbp,nbp}\} \to \{\text{r,TV}\}\}$.

Exercises IV (cont.)

Recall Exercise I: normal blood pressure (nbp), high blood pressure (hbp), watches TV (TV), eats unhealthy food (uf), runs regularly (r).

-	**nbp**	**hbp**	**TV**	**uf**	**r**
A	X				X
B	X			X	X
C		X	X	X	
D		X		X	
E	X				

Reduce the following theory into a non-redundant one.

$$T = \{\{\text{r}\} \to \{\text{nbp}\}, \{\text{TV, uf}\} \to \{\text{hbp}\}, \{\text{TV}\} \to \{\text{uf}\},$$
$$\{\text{TV}\} \to \{\text{hbp}\}, \{\text{hbp,TV}\} \to \{\text{uf}\}, \{\text{nbp,uf}\} \to \{\text{r}\}, \{\text{hbp}\} \to \{\text{uf}\},$$
$$\{\text{uf,r}\} \to \{\text{nbp}\}, \{\text{nbp,TV}\} \to \{\text{r}\}, \{\text{hbp,nbp}\} \to \{\text{r,TV}\}\}.$$

Some useful links

More to come but...

- https://latviz.loria.fr/
- https://upriss.github.io/fca/fca.html
- Binary data: https://github.com/smartfca/caspailleur/
- Complex patterns with FCA (Part II): https://smartfca.github.io/paspailleur/

Functional dependency: For many-valued attributes X, Y in complete many-valued context (G, M, W, J), Y is **functionally dependent** of X, $X \rightarrow_{fd} Y$, if for every pair $g, h \in G$:

$$(\forall m \in X \ m(g) = m(h)) \Rightarrow (\forall n \in Y \ n(g) = n(h)).$$

Example: Many-valued context $K = (G, M, W, J)$, where $b \rightarrow_{fd} c$ is **valid**

G \ M	a	b	c
1	a_1	b_1	c_1
2	a_2	b_1	c_1
3	a_1	b_2	c_2
4	a_2	b_3	c_2

Example: Many-valued context $K = (G, M, W, J)$, where $b \rightarrow_{fd} c$ is **not valid**

G \ M	a	b	c
1	a_1	b_1	c_1
2	a_2	b_1	c_2
3	a_1	b_2	c_2
4	a_2	b_3	c_2

Functional Dependencies as Implications

Proposition: For $\mathbb{K} = (G, M, W, J)$, consider $\mathbb{K}_N := (\mathcal{P}_2(G), M, I)$ where $\mathcal{P}_2(G)$ is the set of pairs of different objects from G, and I is defined by

$$\{g, h\}\, I\, m \Leftrightarrow m(g) = m(h).$$

Then $X \rightarrow_{fd} Y$ holds in $\mathbb{K}$ **iff** the implication $X \rightarrow Y$ holds in $\mathbb{K}_N$.

Example: Consider the following many-valued context $\mathbb{K}$:

G \ M	a	b	c	d
1	α	α	β	β
2	α	γ	α	α
3	δ	α	δ	α
4	α	γ	α	δ

and the following formal context $\mathbb{K}_N = (G, M, I)$:

G \ M	a	b	c	d
{1,2}	×			
{1,3}		×		
{1,4}	×			
{2,3}				×
{2,4}	×	×	×	
{3,4}				

NB: The D-G basis of $\mathbb{K}_N = (G, M, I)$ is the minimal cover of FDs of $\mathbb{K}$.

Conversely: For any $\mathbb{K} = (G, M, I)$, there is $\mathbb{K}_W = (G, M, W, J)$ such that...

Functional Dependencies as Implications

Proposition: For $\mathbb{K} = (G, M, W, J)$, consider $\mathbb{K}_N := (\mathcal{P}_2(G), M, I)$ where $\mathcal{P}_2(G)$ is the set of pairs of different objects from G, and I is defined by

$$\{g, h\}\, I\, m \Leftrightarrow m(g) = m(h).$$

Then $X \to_{fd} Y$ holds in $\mathbb{K}$ **iff** the implication $X \to Y$ holds in $\mathbb{K}_N$.

Example: Consider the following many-valued context $\mathbb{K}$:

G \ M	a	b	c	d
1	α	α	β	β
2	α	γ	α	α
3	δ	α	δ	α
4	α	γ	α	δ

and the following formal context $\mathbb{K}_N = (G, M, I)$:

G \ M	a	b	c	d
{1,2}	×			
{1,3}		×		
{1,4}	×			
{2,3}				×
{2,4}	×	×	×	
{3,4}				

NB: The D-G basis of $\mathbb{K}_N = (G, M, I)$ is the minimal cover of FDs of $\mathbb{K}$.

Conversely: For any $\mathbb{K} = (G, M, I)$, there is $\mathbb{K}_W = (G, M, W, J)$ such that...

Functional Dependencies as Implications

Proposition: For $\mathbb{K} = (G, M, W, J)$, consider $\mathbb{K}_N := (\mathcal{P}_2(G), M, I)$ where $\mathcal{P}_2(G)$ is the set of pairs of different objects from G, and I is defined by

$$\{g, h\}\, I\, m \Leftrightarrow m(g) = m(h).$$

Then $X \to_{fd} Y$ holds in $\mathbb{K}$ **iff** the implication $X \to Y$ holds in $\mathbb{K}_N$.

Example: Consider the following many-valued context $\mathbb{K}$:

G \ M	a	b	c	d
1	α	α	β	β
2	α	γ	α	α
3	δ	α	δ	α
4	α	γ	α	δ

and the following formal context $\mathbb{K}_N = (G, M, I)$:

G \ M	a	b	c	d
{1,2}	×			
{1,3}		×		
{1,4}	×			
{2,3}				×
{2,4}	×	×	×	
{3,4}				

NB: The D-G basis of $\mathbb{K}_N = (G, M, I)$ is the minimal cover of FDs of $\mathbb{K}$.

Conversely: For any $\mathbb{K} = (G, M, I)$, there is $\mathbb{K}_W = (G, M, W, J)$ such that...

References

- W. W. Armstrong. Dependency structures in data base relationships. IFIP Congress, Geneva, Switzerland, 1974, pp. 580–583.
- J. Baixeries, A. Napoli, M. Kaytoue. Characterizing functional dependencies in formal concept analysis with pattern structures, Annals of Mathematics and Artificial Intelligence, 72:129-149, 2014.
- S.O. Kuznetsov, S.A. Obiedkov, *Some Decision and Counting Problems of the Duquenne-Guigues Basis of Implications*. Discr. Appl. Math., vol. 156, no. 11, pp. 1994–2003 (2008).
- F. Distel, B. Sertkaya, *On the complexity of enumerating pseudo-intents*. Discr. Appl. Math. vol. 159(6), pp. 450-466 (2011).
- M.A.Babin, S.O.Kuznetsov, *Computing premises of a minimal cover of functional dependencies is intractable.* Discr. Appl. Math., vol. 161(6), pp. 742-749 (2013).
- D. Maier. The Theory of Relational Databases. CS Press, 1983
- C.J. Date. Database Design and Relational Theory: Normal Forms and All That Jazz. Apress, 2019.

Association rules & hypotheses

Association rules

Idea: Association rules as "partial" FCA implications [Agrawal et al. 90's]

More precisely: An **association rule** of (G, M, I) is an exp. $A \to_{c,s} B$, where

- $c = \frac{|(A \cup B)'|}{|A'|} \in [0, 1]$, called **confidence**, $\mathrm{conf}(A \to B)$;
- $s = \frac{|(A \cup B)'|}{|G|} \in [0, 1]$, called **support**, $\mathrm{supp}(A \to B)$.

Further metrics of interest: *Lift*

$$\mathrm{lift}(A \to B) = \frac{\mathrm{conf}(A \to B)}{\mathrm{supp}(B)}$$

Question: What is a minimal representation of the set of association rules, from which all association rules can be obtained using "admissible transformations"?

Association rules

Idea: Association rules as "partial" FCA implications [Agrawal et al. 90's]

More precisely: An **association rule** of (G, M, I) is an exp. $A \to_{c,s} B$, where

- $c = \frac{|(A \cup B)'|}{|A'|} \in [0, 1]$, called **confidence**, $\mathrm{conf}(A \to B)$;
- $s = \frac{|(A \cup B)'|}{|G|} \in [0, 1]$, called **support**, $\mathrm{supp}(A \to B)$.

Further metrics of interest: *Lift*

$$\mathrm{lift}(A \to B) = \frac{\mathrm{conf}(A \to B)}{\mathrm{supp}(B)}$$

Question: What is a minimal representation of the set of association rules, from which all association rules can be obtained using "admissible transformations"?

Covers of association rules

Idea: Consider $A \rightarrow_{c,s} B$. Under fixed c and support s we try to **reduce** its premise and **increase** its conclusion

1. Decreasing premises: For fixed c and s we can decrease the premise to $D \subseteq A$ such that $(D \cup B)' = (A \cup B)' = A' \cap B' = D' \cap B'$, **i.e.**, $D' = A' = A'''$. **Thus:** minimal D is by definition a minimal generator of A'', i.e. $D \in \text{mingen}(A'')$.

Recall 1: $D \subseteq M$ is a **generator** of $B = B'' \subseteq M$ if $D \subseteq B$ and $D'' = B = B''$.

Recall 2: $D \subseteq M$ is a **minimal generator** if for every $E \subset D$, $E'' \neq D'' = B''$.

Covers of association rules (cont.)

2. Increasing conclusion. Conclusion B can be increased by Δ such that $(A \cup B)' = (A \cup B \cup \Delta)' = (A \cup B)' \cap \Delta'$, which is possible **only when** $(A \cup B)' \subseteq \Delta'$.

Or equiv.: to $A \cup B \to \Delta$ and to $\Delta \subseteq (A \cup B)''$. **Thus:** the conclusion can be increased up to $(A \cup B)''$.

Exercise: Show that these transformations do not decrease confidence nor support!

Conclusion 1: The set $CP(K) = \{D \to (A \cup B)'' \mid D \in \text{ mingen } (A'')\}$ is a cover of the set of all association rules.

Conclusion 2: All other rules can be otained by admissible transformations on rules from $CP(K)$, **i.e.**, $CP(K)$ is a cover of association rules.

Covers of association rules (cont.)

2. Increasing conclusion. Conclusion B can be increased by Δ such that $(A \cup B)' = (A \cup B \cup \Delta)' = (A \cup B)' \cap \Delta'$, which is possible **only when** $(A \cup B)' \subseteq \Delta'$.

Or equiv.: to $A \cup B \to \Delta$ and to $\Delta \subseteq (A \cup B)''$. **Thus:** the conclusion can be increased up to $(A \cup B)''$.

Exercise: Show that these transformations do not decrease confidence nor support!

Conclusion 1: The set $CP(K) = \{D \to (A \cup B)'' \mid D \in \text{ mingen } (A'')\}$ is a cover of the set of all association rules.

Conclusion 2: All other rules can be otained by admissible transformations on rules from $CP(K)$, **i.e.**, $CP(K)$ is a cover of association rules.

Bases of association rules

Consider an assoc. rule of the form $D \to (A \cup B)''$, where $D \in \text{mingen}(A'')$. **In the concept lattice** this rule corresponds to a path (A', A'') to the concept $((A \cup B)', (A \cup B)'')$.

NB: If (A', A'') is not an upper cover $((A \cup B)', (A \cup B)'')$, **then** there is a (E', E'') such that $(A', A'') \succ (E', E'') \succ ((A \cup B)', (A \cup B)'')$.

Consider $D \to E''$ ($D \in \text{mingen}(A'')$), and $F \to (A \cup B)''$ ($F \in \text{mingen}(E'')$). **Then**

1. the confidence of the first rule is $c_1 = \frac{|E'|}{|A'|}$,
2. the confidence of the second rule is $c_2 = \frac{|(A \cup B)'|}{|E'|}$, and
3. the confidence of $D \to (A \cup B)''$, where $D \in \text{mingen}(A'')$ is

$$c = \frac{|(A \cup B)'|}{|A'|} = \frac{|E'|}{|A'|} \cdot \frac{|(A \cup B)'|}{|E'|} = c_1 \cdot c_2.$$

Bases of association rules

Consider an assoc. rule of the form $D \to (A \cup B)''$, where $D \in \text{mingen}(A'')$. **In the concept lattice** this rule corresponds to a path (A', A'') to the concept $((A \cup B)', (A \cup B)'')$.

NB: If (A', A'') is not an upper cover $((A \cup B)', (A \cup B)'')$, **then** there is a (E', E'') such that $(A', A'') \succ (E', E'') \succ ((A \cup B)', (A \cup B)'')$.

Consider $D \to E''$ ($D \in \text{mingen}(A'')$), and $F \to (A \cup B)''$ ($F \in \text{mingen}(E'')$). **Then**

1. the confidence of the first rule is $c_1 = \frac{|E'|}{|A'|}$,
2. the confidence of the second rule is $c_2 = \frac{|(A \cup B)'|}{|E'|}$, and
3. the confidence of $D \to (A \cup B)''$, where $D \in \text{mingen}(A'')$ is

$$c = \frac{|(A \cup B)'|}{|A'|} = \frac{|E'|}{|A'|} \cdot \frac{|(A \cup B)'|}{|E'|} = c_1 \cdot c_2.$$

Bases of association rules (cont.)

Hence the cover of the set of association rules can be made even smaller by restricting it to

$$\mathcal{A} = \{F \to (E'' \setminus F'') \mid F \subseteq M, F \in \text{mingen}(F''), \textbf{ and } (F', F'') \succ (E', E'')\}$$

NB1: They correspond to the arcs of the lattice diagram!

NB2: Support and confidence of other rules from the cover can be obtained by multiplying supports along the respective paths in the lattice

NB3: To minimize the cover of association rules, i.e., a **basis**, keep only those rules from $CP(K)$ that correspond to edges in a spanning tree of the lattice

Bases of association rules (cont.)

Hence the cover of the set of association rules can be made even smaller by restricting it to

$$\mathcal{A} = \{F \to (E'' \setminus F'') \mid F \subseteq M, F \in \text{mingen}(F''), \textbf{ and } (F', F'') \succ (E', E'')\}$$

NB1: They correspond to the arcs of the lattice diagram!

NB2: Support and confidence of other rules from the cover can be obtained by multiplying supports along the respective paths in the lattice

NB3: To minimize the cover of association rules, i.e., a **basis**, keep only those rules from $CP(K)$ that correspond to edges in a spanning tree of the lattice

Recall: Finding association rules...

Task: Find all "frequent" ($s >$threshold) association rules with $c > \theta$.

Solution steps:

1. Find all frequent "closed itemsets" (frequent intents)
2. For each frequent intent B find all its maximal subintents $A_1, \ldots, A_n$
3. Retain only those A_i for which $\text{conf}(A_i \to B) \geq \theta$
4. Find minimal generators of the remaining A_i, compose rules of the form $mingen(A_i) \to B$.

Luxenburger basis:

- Spanning tree of the concept lattice diagram
  (confident rules btw neigh. closed intents, w.r.t. supp. & conf.)
- Duquenne-Guigues implication base

Recall: Finding association rules...

Task: Find all "frequent" ($s >$threshold) association rules with $c > \theta$.

Solution steps:

1. Find all frequent "closed itemsets" (frequent intents)
2. For each frequent intent B find all its maximal subintents $A_1, \ldots, A_n$
3. Retain only those A_i for which $\text{conf}(A_i \to B) \geq \theta$
4. Find minimal generators of the remaining A_i, compose rules of the form $mingen(A_i) \to B$.

Luxenburger basis:

- Spanning tree of the concept lattice diagram
  (confident rules btw neigh. closed intents, w.r.t. supp. & conf.)
- Duquenne-Guigues implication base

References

- R. Agrawal, H. Mannila, R. Srikant, H. Toivonen, A. I. Verkamo, Fast Discovery of Association Rules, in Advances in Knowledge Discovery and Data Mining, AAAI Press/MIT Press, 1996.
- N. Pasquier, Y. Bastide, R. Taouil, L. Lakhal, Discovering Frequent Closed Itemsets for Association Rules, ICDT'99 Proceedings, LNCS 1540, Springer, pages 398–416, 1999.
- Y. Bastide, R. Taouil, N. Pasquier, G. Stumme, L. Lakhal, Mining frequent patterns with counting inference, ACM SIGKDD Explorations, 2(2):66–75, 2000.
- S.O.Kuznetsov, Complexity of Learning in Concept Lattices from Positive and Negative Examples. Discr. Appl. Math., **142**, pp. 111–125 (2004).
- D. I. Ignatov, Introduction to Formal Concept Analysis and Its Applications in Information Retrieval and Related Fields. CoRR abs/1703.02819